

第4章

单(私)钥密码体制

单钥加密体制也称为私钥加密体制 (Secret Key Cryptosystem)。由于通信双方采用的密钥相同, 所以人们通常也称其为对称加密体制 (Symmetric Cryptosystem)。

对于单钥加密体制来说, 可以按照其加解密运算的特点, 将其分为流密码 (Stream Cipher) 和分组密码 (Block Cipher)。涉及流密码和分组密码的理论和内容非常多, 很多书中将流密码和分组密码分章讨论。由于对流密码和分组密码的理论上的描述已经超出了本书的范围, 所以本章将流密码和分组密码合为一章讨论。本章主要介绍流密码和分组密码的基本理论, 以及有代表性的分组密码算法, 并对其具体的技术问题进行讨论。

4.1

密码体制的定义

密码体制的语法定义如下:

- 明文消息空间 M : 某个字母表上的串集。
- 密文消息空间 C : 可能的密文消息集。
- 加密密钥空间 K : 可能的加密密钥集; 解密密钥空间 K' : 可能的解密密钥集。
- 有效的密钥生成算法 $\zeta: N \rightarrow K \times K'$ 。
- 有效的加密算法 $E: M \times K \rightarrow C$ 。
- 有效的解密算法 $D: C \times K' \rightarrow M$ 。

对于整数 l' , $\zeta(l')$ 输出长为 l 的密钥对 $(ke, kd) \in K \times K'$,

对于 $ke \in K$ 和 $m \in M$, 将加密变换表示为

$$c = E_{ke}(m)$$

读做“ c 是 m 在密钥 ke 下的加密”; 将解密变换表示为

$$m = D_{kd}(c)$$

读做“ m 是 c 在密钥 kd 下的解密”。对于所有的 $m \in M$ 和所有的 $ke \in K$, 一定存在 $kd \in K'$:

$$D_{kd}(E_{ke}(m)) = m \quad (4-1)$$

在本书的其余各章, 除了文献上已经习惯使用不同记号的地方, 将使用这个构造性的记号集来表示抽象的密码体制。图 4-1 是密码体制的图示。

现将密码体制的构成空间和算法符号应用于既使用私钥又使用公钥 (公钥密码体制将在第 5 章中介绍) 的密码体制。在单钥密码体制中, 加密和解密使用同样的密钥, 加

密消息的人必须与即将收到已加密消息并对其解密的人分享加密密钥。 $k_d = k_e$ 的情况给了单钥密码体制另一个名字：对称密码体制（Symmetric Cryptosystem）。在公钥密码体制中，加密和解密使用不同的密钥，对于每个 $k_e \in K$ ，存在 $k_d \in K'$ ，这两个密钥不同，但互相匹配；加密密钥 k_e 不必保密， k_e 的拥有者可以使用相匹配的私钥 k_d 来解密在 k_e 下加密过的密文。 $k_d \neq k_e$ 的情况给了公钥密码体制另一个名字：非对称密码体制（Asymmetric Cryptosystem）。

1883 年，Kerchoffs 列了一个设计密码要求必备的条件表[Menezes 等 1997]。在 Kerchoffs 列表中，有一条已经发展为被广泛认可的约定，称为 Kerchoffs 原理。

现代密码分析的标准假设是攻击者可以获知密码算法、密钥长度以及密文。既然敌手最终可以获得这些信息，那么评估密码强度时最好不要依赖这些信息的保密性。

结合香农对密码体制的语义描述和 Kerchoffs 原理，可以对好的密码体制做如下总结：

- 算法 E 和 D 不包含秘密的成分或设计部分。
- E 将有意义的消息相当均匀地分布在整个密文消息空间中；甚至可以由 E 的某些随机的内部运算来获得随机的分布。
- 使用正确的密钥， E 和 D 是实际有效的。
- 不使用正确的密钥，要由密文恢复出相应的明文是一个由密钥参数的大小唯一决定的困难问题，通常取长为 s 的密钥，使得解这个问题所要求计算资源的量级超过 $p(s)$ ， p 是任意多项式。

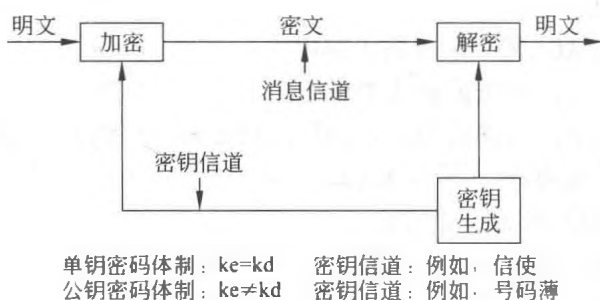


图 4-1 密码体制

注意：希望密码体制具有以上这些性质对于现代密码体制的应用来说已经不够了，通过对密码体制的研究，将归纳出一些更为严格的要求。

4.2 古典密码

古典密码是密码学的渊源，这些密码大都比较简单，可用手工或机械操作实现加解密，现在已很少采用了。然而，研究这些密码的原理，对于理解、构造和分析现代密码都是十分有益的。

4.2.1 代换密码

在代换密码 (Substitution Cipher) 中, 加密算法 $E_k(m)$ 是一个代换函数, 它将每一个 $m \in M$ 代换为相应的 $c \in C$, 代换函数的参数是密钥 k , 解密算法 $D_k(c)$ 只是一个逆代换。通常, 代换可由映射 $\pi: M \rightarrow C$ 给出, 而逆代换恰是相应的逆映射 $\pi^{-1}: C \rightarrow M$ 。

1. 简单的代换密码

例 4-1 简单的代换密码。令 $M = C = Z_{26}$, 所包含元素表示为 $A=0, B=1, \dots, Z=25$ 。将加密算法 $E_k(m)$ 定义为下面的 Z_{26} 上的一个置换

$$\begin{pmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 \\ 21 & 12 & 25 & 17 & 24 & 23 & 19 & 15 & 22 & 13 & 18 & 3 & 9 \\ 13 & 14 & 15 & 16 & 17 & 18 & 19 & 20 & 21 & 22 & 23 & 24 & 25 \\ 5 & 10 & 2 & 8 & 16 & 11 & 14 & 7 & 1 & 4 & 20 & 0 & 6 \end{pmatrix}$$

那么相应的解密算法 $D_k(c)$ 为

$$\begin{pmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 \\ 24 & 21 & 15 & 11 & 22 & 13 & 25 & 20 & 16 & 12 & 14 & 18 & 1 \\ 13 & 14 & 15 & 16 & 17 & 18 & 19 & 20 & 21 & 22 & 23 & 24 & 25 \\ 9 & 19 & 7 & 17 & 3 & 10 & 6 & 23 & 0 & 8 & 5 & 4 & 2 \end{pmatrix}$$

明文消息

proceed meeting as agreed

加密为下面的密文消息 (空间并不改变)

cqkzyyrjyyowftvlvtqyyr

在这个简单的代换密码的例子里, 消息空间 M 和 C 都是字母表 Z_{26} , 换句话说, 一个明文或密文消息是字母表中的一个单个字符。由于这个原因, 明文消息串 proceedmeetingasagreed 并不是单个的消息, 而是包含了 22 个消息, 同样, 密文消息串 cqkzyyrjyyowftvlvtqyyr 也包含 22 个消息。密码的密钥空间大小为 $26! > 4 \times 10^{26}$, 与消息空间的大小相比是非常大的。然而, 事实上这种密码是非常弱的: 每一个明文字符被加密成唯一的密文字符。这一弱点致使这种密码对于称为频度分析的一种密码分析技术来说, 是相当脆弱的, 频度分析揭示出一个事实, 就是自然语言包含大量的冗余。

历史上出现过几种特殊的简单代换密码, 最简单且最著名的密码称为移位密码。在移位密码中, $K = M = C$, 令 $N = \#M$, 则加密和解密映射定义为

$$\begin{cases} E_k(m) \leftarrow m + k \pmod{N} \\ D_k(c) \leftarrow c - k \pmod{N} \end{cases} \quad (4-2)$$

其中 $m, c, k \in Z_N$ 。当 M 为拉丁字母表的大写字母时, 也就是 $M = Z_{26}$, 移位密码也称为凯撒密码, 这是因为 Julius Caesar 使用了该密码当 $k=3$ 时的情形 [Denning 1982]。

如果 $\gcd(k, N)=1$, 那么对每个 $m < N$:

$$km(\text{mod } N)$$

可取遍整个消息空间 Z_N ，因此对于这样的 k 和 $m, c < N$

$$\begin{cases} E_k(m) \leftarrow km(\text{mod } N) \\ D_k(c) \leftarrow k^{-1}c(\text{mod } N) \end{cases} \quad (4-3)$$

给出了一种简单代换密码。同理，

$$k_1m + k_2(\text{mod } N)$$

也可以定义一种称为仿射密码的简单代换密码：

$$\begin{cases} E_k(m) \leftarrow k_1m + k_2(\text{mod } N) \\ D_k(c) \leftarrow k_1^{-1}(c - k_2)(\text{mod } N) \end{cases} \quad (4-4)$$

不难看出，利用 K 中密钥与 M 中消息之间的不同算术运算可以设计不同的简单代换密码，这些密码称为单表密码（Monoalphabetic Cipher）：对于一个给定的加密密钥，明文消息空间中的每一元素将被代换为密文消息空间中的唯一元素。因此，单表密码不能抵抗频度分析攻击。

然而，由于简单代换密码的简易性，它们已经被广泛应用于现代单钥加密算法中。在后面的两节中，将介绍简单代换密码在数据加密标准（DES）和高级加密标准（AES）中所起到的核心作用。几个简单密码算法的结合可以产生一个安全的密码算法，这一点已经得到大家的认可，这就是简单密码仍被广泛应用的原因。简单代换密码在密码协议上也有广泛的应用。

2. 多表密码

如果 P 中的明文消息元可以代换为 C 中的许多、可能是任意多的密文消息元，这种代换密码就称为多表密码（Polyalphabetic Cipher）。

由于维吉尼亚密码（Vigenère Cipher）是多表密码中最知名的密码，所以下面将以它为例来说明多表密码。

维吉尼亚密码是基于串的代换密码：密钥是由多于一个的字符所组成的串。令 m 为密钥长度，那么明文串被分为 m 个字符的小段，也就是说，每一小段是 m 个字符的串，可能的例外就是串的最后一小段不足 m 个字符。加密算法的运算同于密钥串和明文串之间的移位密码，每次的明文串都使用重复的密钥串。解密同于移位密码的解密运算。

例 4-2 维吉尼亚密码。令密钥串是 gold，利用编码规则 $A=0, B=1, \dots, Z=25$ ，这个密钥串的数字表示是 (6, 14, 11, 3)。明文串

proceed meeting as agreed

的维吉尼亚加密运算如下，这种运算就是逐字符模 26 加：

15	17	14	2	4	4	3	12	4	4	19
6	14	11	3	6	14	11	3	6	14	11
21	5	25	5	10	18	14	15	10	18	4
8	13	6	0	18	0	6	17	4	4	3
3	6	14	11	3	6	14	11	3	6	14
11	19	20	11	21	6	20	2	7	10	17

因此密文串是

vfzfkso pkseltu lv guchkr

其他著名的多表密码还包括书本密码(也称做 Beale 密码)和 Hill 密码,它们的密钥串是已协商好的书中的原文。有关这些代换密码的详细描述请参考相关文献[Denning 1982; Stinson 1995]。

3. 弗纳姆密码和一次一密

弗纳姆密码是最简单的密码体制之一。若假定消息是长为 n 的比特串

$$m = b_1 b_2 \cdots b_n \in \{0,1\}^n$$

那么密钥也是长为 n 的比特串

$$k = k_1 k_2 \cdots k_n \in_U \{0,1\}^n$$

(这里注意到符号“ $\in_U$ ”表示均匀随机地选取 k)。一次加密一比特,通过将每个消息比特和相应的密钥比特进行比特 XOR (异或) 运算来得到密文串 $c = c_1 c_2 \cdots c_n$

$$c_i = b_i \oplus k_i$$

$1 \leq i \leq n$, 这里运算 $\oplus$ 定义为

$\oplus$	0	1
0	0	1
1	1	0

因为 $\oplus$ 是模 2 加,所以减法等于加法,因此解密与加密相同。

考虑 $M = C = K = \{0,1\}^*$, 则弗纳姆密码是代换密码的特例。如果密钥串只使用一次,那么弗纳姆密码就是一次一密加密体制。一次一密弗纳姆密码提供的保密性是在信息理论安全性的意义上的,或者说,是无条件的。理解这种安全性的一种简单方法如下:

如果密钥 k 等于 $c \oplus m$ (逐比特模 2 加),由于任意 m 能够产生 c ,所以密文消息串 c 不能提供给窃听者关于明文消息串 m 的任何信息。

一次一密弗纳姆密码也称为一次一密钥密码。原则上,只要加密密钥的使用满足安全代换密码必须满足的两个条件[Mao 2004],那么任何代换密码都是一次一密密码。然而习惯上只有使用逐比特异或运算的密码才称为一次一密密码。

与其他代换密码(例如使用模 26 加的移位密码)相比,逐位异或运算(模 2 加)在电子电路中更容易实现,因为这个原因,逐位异或运算被广泛应用在现代单钥加密算法的设计中。现代密码 DES、AES 和我国设计的祖冲之密码算法(ZUC)均使用了逐位异或运算。

一次一密钥类型也被广泛应用在密码学协议中。

4.2.2 换位密码

通过重新排列消息中元素的位置而不改变元素本身来变换一个消息的密码称做换位密码(也称做置换密码)。换位密码是古典密码中除代换密码外的重要一类,它广泛应用于现代分组密码的构造。

考虑明文消息中的元素是 Z_{26} 中的字符时的情形, 令 b 为一固定的正整数, 它表示消息分组的大小, $P=C=(Z_{26})^b$, 而 K 是所有的置换, 也就是 $(1,2,\dots,b)$ 的所有重排。

那么因为 $\pi \in K$, 置换 $\pi=(\pi(1),\pi(2),\dots,\pi(b))$ 是一个密钥。对于明文分组 $(x_1,x_2,\dots,x_b) \in P$, 这个换位密码的加密算法是

$$E_{\pi}(x_1,x_2,\dots,x_b)=(x_{\pi(1)},x_{\pi(2)},\dots,x_{\pi(b)})$$

令 π^{-1} 表示 π 的逆, 也就是 $\pi^{-1}(\pi(i))=i, i=1,2,\dots,b$, 那么这个换位密码相应的解密算法是

$$D_{\pi}(y_1,y_2,\dots,y_b)=(y_{\pi(1)}^{-1},y_{\pi(2)}^{-1},\dots,y_{\pi(b)}^{-1})$$

对于长度大于分组长度 b 的消息, 该消息可分成多个分组, 然后逐分组重复同样的过程。

既然对于消息分组的长度 b , 共有 $b!$ 种不同的密钥, 因此一个明文消息分组能够变换加密为 $b!$ 种可能的密文, 然而由于字母本身并未改变, 换位密码对于抗频度分析技术也是相当脆弱的。

例 4-3 换位密码。令 $b=4$, $\pi=(\pi(1),\pi(2),\pi(3),\pi(4))=(2,4,1,3)$, 那么明文消息

proceed meeting as agreed

首先分为 6 个分组, 每个分组 4 个字符:

proc eedm eeti ngas agre ed

然后可以变换-加密成下面的密文

rcpoemedeietsgsnagearde

注意到明文的最后一个短分组 ed 实际上填充成了 ed□□, 然后加密成 d□e□, 再从密文分组中删掉补上的空格。解密密钥是

$$\pi^{-1}=(\pi(1))^{-1},\pi(2))^{-1},\pi(3))^{-1},\pi(4))^{-1})=(2^{-1},4^{-1},1^{-1},3^{-1})$$

最终的缩短密文分组 de 只包含两个字母说明了在相应的明文分组中没有字符与 3^{-1} 和 4^{-1} 的位置相匹配, 因此在解密过程正确执行以前, 应该将空格重新插入到缩短的密文分组中它们原来的位置上, 以便将分组恢复成添加空格的形式 d□e□。

注意到对于最后的明文分组较短的情况 (比如例 4-3 的情形), 由于添加的字符暴露了所用密钥的信息, 因此在密文消息中不要留下例如□这样的添加字符。

4.2.3 古典密码的安全性

首先指出, 古典密码有两个基本工作原理: 代换和换位。它们仍是构造现代对称加密算法的最重要的核心技术。后面介绍代换和换位密码在两个重要的现代对称加密算法 DES 和 AES 中的结合。

考虑基于字符的代换密码, 因为明文消息空间就是字母表, 每个消息就是字母表中的一个字符, 加密就是逐字符地将每一明文字符代换为一个密文字符, 代换取决于密钥。在加密一个长字符串时, 如果密钥是固定的, 那么在明文消息中同一个字符将被加密成

密文消息中一个固定的字符。

众所周知,自然语言中的字符有稳定的频度,自然语言中的字符频度分布知识为密码分析(由已知密文消息发现明文或加密密钥信息的技术)提供了线索,例 4-1 表明了这一情形,该例中的字符 y 在密文消息中高频出现,这表明一定有一个固定的字符在相应的明文消息中以相同的频率出现(事实上这个字符就是 e,在英语中它是一个高频出现的字符)。简单代换密码不能隐藏基于自然语言的信息,基于字符频度研究的密码分析技术的详细内容可参阅密码学的有关教材[Denning 1982; Menezes 等 1997]。

表密码和换位密码都比简单代换密码安全,但是,如果密钥很短而消息很长,那么就有各种各样的密码分析技术能够攻破这样的密码。

然而如果密钥的使用满足了某些条件,那么古典密码,甚至是简单代换密码也可以是非常安全的。事实上,在正确地使用了密钥以后,简单代换密码可以广泛应用于密码体制和协议。

4.3

流密码的基本概念

流密码是密码体制中的一个重要体制,也是手工和机械密码时代的主流。20 世纪 50 年代,由于数字电子技术的发展,使密钥流可以方便地利用以移位寄存器为基础的电路来产生,这促使线性和非线性移位寄存器理论迅速发展,加上有效的数学工具,如代数和谱分析理论的引入,使得流密码理论迅速发展和走向较成熟的阶段。同时由于它实现简单和速度上的优势,以及没有或只有有限的错误传播,使流密码在实际应用中,特别是在专用和机密机构中仍保持优势。已提出多种类型的流密码,但大多是以硬件实现的专用算法,目前还无标准化的流密码算法。本章将对流密码的基本理论和算法进行介绍,同时也讨论一些最近提出的新型流密码,如混沌密码序列和量子密码。有关密码的综述可参阅[Rueppel 1986a, 1992]。

流密码是将明文划分成字符(如单个字母),或其编码的基本单元(如 0, 1 数字),字符分别与密钥流作用进行加密,解密时以同步产生的同样的密钥流实现,其基本框图如图 4-2 所示。图中,KG 为密钥流生成器, k_i 为初始密钥。流密码强度完全依赖于密钥流产生器所生成序列的随机性(randomness)和不可预测性(unpredictability)。其核心问题是密钥流生成器的设计。保持收发两端密钥流的精确同步是实现可靠解密的关键技术。

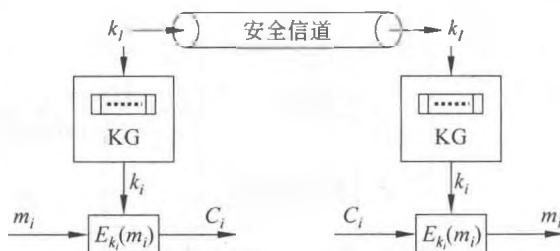


图 4-2 流密码原理框图

4.3.1 流密码框图和分类

令 $m = m_1m_2 \cdots m_i$ 是待加密消息流，其中 $m_i \in M$ 。密文流 $c = c_1c_2 \cdots c_i \cdots = E_{k_1}(m_1)E_{k_2}(m_2) \cdots E_{k_i}(m_i) \cdots$ ， $c_i \in C$ 。其中 $\{k_i\}(i \geq 0)$ 是密钥流。若它是一个完全随机的非周期序列，则可用它实现一次一密体制。但这需要有限存储单元和复杂的逻辑函数 f 。实用中的流密码大多采用有限存储单元和确定性算法，因此可用有限状态自动机（Finite State Automaton, FSA）来描述。如图 4-3 所示。

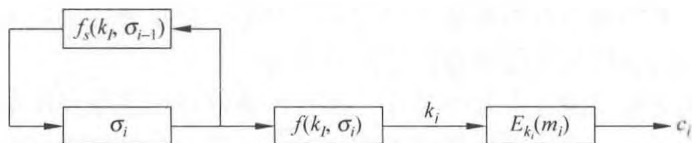


图 4-3 KG 的有限状态自动机描述

$$c_i = E_{k_i}(m_i) \quad (4-5)$$

$$m_i = D_{k_i}(c_i) \quad (4-6)$$

$$k_i = f(k_i, \sigma_i) \quad (4-7)$$

$$\sigma_i = f_s(k_i, \sigma_{i-1}) \quad (4-8)$$

是第 i 时刻密钥流生成器的内部状态，以存储单元的存数矢量描述； k_i 是初始密钥， f 是输出函数， f_s 是状态转移函数。若

$$c_i = E_{k_i}(m_i) = m_i \oplus k_i \quad (4-9)$$

则称这类密码为加法流密码。

若 σ_i 与明文消息无关，则密钥流将独立于明文，称此类为同步流密码（Synchronous Stream Cipher, SSC），如图 4-4 所示。对于明文而言，这类加密变换是无记忆的，但它是时变的。因为同一明文字符在不同时刻，由于密钥不同而被加密成不同的密文字符。此类密码只要收发两端的密钥流生成器的初始密钥 k_i 和初始状态相同，输出的密钥就一样。因此，只有保持两端精确同步才能正常工作，一旦失步就不能正确解密，必须等到重新同步后才能恢复正常工作。这是其主要缺点。但由于其对失步的敏感性，使得系统在有窜扰者进行注入、删除、重放等主动攻击时异常敏感而有利于检测。此类体制的优点是传输中出现的一些偶然错误，只影响相应位的恢复消息，没有差错传播（Error Propagation）。许多古典密码，如周期为 d 的维吉尼亚密码、转轮密码、滚动密钥密码、弗纳姆密码等，都是同步型流密码。同步型流密码在失步后如何重新同步是一个重要技

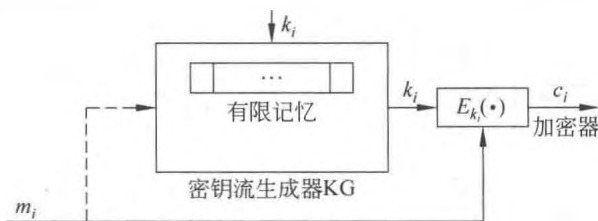


图 4-4 同步和自同步流密码

术研究课题, 处理不好会严重影响系统的安全性。

另一类是自同步流密码 (Self-Synchronous Stream Cipher, SSSC)。如图 4-4 中虚线所示。其 σ_i 依赖于 (k_i, σ_{i-1}, m_i) , 因而历史地将与 $m_1, m_2, \dots, m_{i-1}$ 有关。这将使密文 c_i 不仅与当前输入 m_i 有关, 而且由于 k_i 对 σ_i 的关系而与以前的输入 $m_1, m_2, \dots, m_{i-1}$ 有关。一般在有限的 n 级存储下将与 $m_1, m_2, \dots, m_{i-1}$ 有关。图 4-5 所示一种有 n 级移位寄存器存储的密文反馈型流密码。每个密文数字将影响以后 n 个输入明文数字的加密结果。此时的密钥流 $k_i = f(k_i, c_{i-n}, c_{i-n+1}, \dots, c_{i-1})$ 。由于 c_i 与 m_i 的关系, k_i 最终要受输入明文数字的影响。这类流密码的密钥流都可由式 (4-10) 表示:

$$k_i = f(k_i, m_{i-n}, m_{i-n+1}, \dots, m_{i-1}) \quad (4-10)$$

其中

$$f: k_i \times M^n \rightarrow k_i \quad (4-11)$$

军事上称这类流密码为密文自密钥 (Ciphertext Autokey) 密码。

自同步流密码传输过程中有一位 (如 c_i 位) 出错, 在解密过程中, 它将在移存器中存活 n 个节拍, 因而会影响其后 n 位密钥的正确性, 相应恢复的明文消息连续 n 位会受到影响。其差错传播是有限的。但这类体制, 收端只要连续正确地收到 n 位密文, 则在相同密钥 k_i 作用下就会产生相同的密钥, 因而它具有自同步能力。这种自恢复同步性使得它对窜扰者的一些主动攻击不像同步流密码体制那样敏感。但它将明文每个字符扩散在密文多个字符中而强化了其抗统计分析的能力。Maurer[1991]给出了自同步流密码的设计方法。如何控制自同步流密码的差错传播以及它对安全性的影响可参阅相关文献。

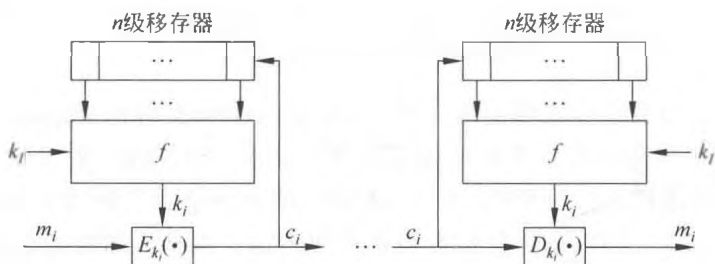


图 4-5 自同步流密码

综上所述, 实际应用中的密钥流都是由有限存储和有限复杂逻辑电路来产生的, 即用有限状态机来实现。一个有限状态机在确定逻辑连接下不可能产生一个真正随机序列, 它迟早要步入周期状态。因而不可能用它来实现一次一密体制。但是可以使这类机器产生的序列周期足够长 (如 1050), 而且其随机性又相当好, 从而可方便地近似实现人们所追求的理想体制。20 世纪 50 年代以来, 以有限自动机为主流的理论和方法得到了迅速发展。近年来虽然出现了不少新的产生密钥流的理论和方法, 如混沌密码、胞元自动机密码、热流密码等, 但在有限精度的数字实现的条件下最终都可归结为用有限自动机来描述。因此, 研究这类序列产生器的理论是流密码研究中最重要基础。

4.3.2 密钥流生成器的结构和分类

Rueppel[1986b]用一个更清楚的框图, 将密钥流生成器分成两个主要组成部分, 即驱

动部分和组合部分，如图 4-6 所示。驱动部分产生控制生成器的状态序列 $S_1, S_2, \dots, S_N$ ，用一个或多个长周期线性反馈移位寄存器构成，它控制生成器的周期和统计特性。非线性组合部分对驱动器各输出序列进行非线性组合，控制和提高生成器输出序列的统计特性、线性复杂度和不可预测性等，以实现 Shannon 提出的扩散和混淆，保证输出密钥流的密码的强度。

为了保证输出密钥流的密码强度，对组合函数 F 有下述要求：

- (1) F 将驱动序列变换为滚动密钥序列，当输入为二元随机序列时，输出也为二元随机序列。
- (2) 对于给定周期的输入序列，构造的 F 使输出序列的周期尽可能大。
- (3) 对于给定复杂度输入序列，构造的 F 输出序列的复杂度尽可能大。
- (4) F 的信息泄漏极小化（从输出难以提取有关密钥流生成器的结构信息）。
- (5) F 应易于工程实现，工作速度高。
- (6) 在需要时， F 易于在密钥控制下工作。

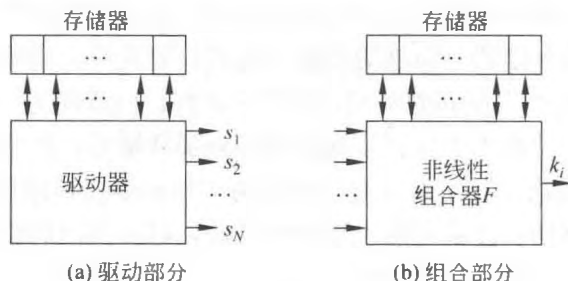


图 4-6 密钥流生成器组成

驱动器一般利用线性反馈移位寄存器（Linear Feedback Shift Register, LFSR），特别是最长或 m 序列产生器实现。非线性反馈移位寄存器（NLFSR）也可作为驱动器，但由于在数学分析上的困难而很少采用。NLFSR 输出序列的密码特性较 LFSR 输出序列要好得多。同样由于分析上的困难性，目前所得结果有限，从而限制了它的应用。

当前密码上广泛应用的非线性序列是图 4-6 所示的由线性序列经非线性组合所产生的密钥流。这实际上是一种非线性前馈（forward）序列生成器。这类序列在较好掌握的线性序列组 $S_1, S_2, \dots, S_N$ 的基础上，利用一些可以用布尔逻辑、谱分析理论等数学工具来设计和控制的非线性组合函数，使其组合输出序列满足密码强度要求。常用的方法有逻辑与、J-K 触发器、多路复用器、钟控、Bent 函数、背包函数等。

4.3.3 密钥流的局部统计检验

对于密钥流生成器输出的密钥序列，必须进行必要的统计检验，以确保密钥序列的伪随机性和安全性。已经设计好的密钥生成器，原则上可以计算其输出的整个周期上的一些伪随机性 G-1~G-3。但由于其输出序列周期都很长，一般在 $10^{17} \sim 10^{40}$ ，因而不可能直接计算，只能利用数理统计方法进行局部伪随机性检验。常用的方法有频度检验、序偶或联码（测定相邻码元的相关性）检验、扑克（图样分布）检验、游程或串长分布检验、

自相关特性检验和局部复杂性检验等。通过这类检验的密钥序列可以在统计上证实其分布的均匀性。但还不能证实其独立性,有一些方法可以演示它没有明显的相关性,一般是利用这些方法来试验直到对其独立性有足够信任。当然,这并不能确保其安全性,因此还要对其密码强度进行估计,需要从其所用非线性函数构造和所具有的密码性质进行分析。有关局部统计检验可参阅有关书刊和标准 [Maurer 1992b; Menezes 等 1997]。

如前所述,密钥流必须具有随机性,同时在收端还应能够同步生成它,否则就不能实现解密。在网络安全系统中,如交互认证协议中 Nonce(一次性随机数)、密钥分配系统的会话密钥等,需要一种一次性且不要求在收端重新同步产生的随机数。对这类随机数生成器的基本要求和密钥流生成器一样,必须满足随机性和不可预测性。由于它们一般较短,所以在实现上与密钥流生成器不太一样,本章后面将介绍生成随机数的一些具体方法。

4.4 快速软、硬件实现的流密码算法

近年来,人们对简化流密码的软硬件实现进行了大量的研究,提出了不少新的易于实现的算法,有些是成功的;有些虽不安全,但在设计思想上有参考价值。有些算法适合硬件实现、有些算法适合软件实现。有些算法则是按兼顾两者的需要来设计的。软件密码的计算量是算法和算法实现质量的函数,一个用硬件实现的好算法,未必在软件实现上也是最佳的。DES 这一在硬件实现上很有效的算法也不例外。所以,寻找适用一般计算机实现的最佳软件算法,也需要精心设计 [Schneier 等 1997]。本节将介绍其中一些有意义的算法。

4.4.1 A5

A5 是欧洲数字蜂窝移动电话系统 (Group Special Mobile, GSM) 中采用的加密算法,用于电话手机到基站线路上的加密。但在链路上的其他段不加密,因此电话公司很容易窃听用户会话。

A5 由法国设计。在 20 世纪 80 年代中期, NATO 内部对 GSM 的加密有过争议,有人认为加密会妨碍出口,而另有些人则认为应当采用强度大的密码进行保护。

A5 由 3 个稀疏本原多项式构成的 LFSR 组成,级数分别为 19、22 和 23,其初态由密钥独立赋值。输出是 3 个 LFSR 输出的异或,采用可变钟控方式,控制位从每个寄存器中间附近选定。若控制位中有两个或 3 个取值为 1,则产生这种位的寄存器移位;若两个或 3 个控制位为 0,则产生这种位的寄存器不移位。显然,在这种工作于停走 (stop/go) 型的相互钟控 (或锁定) 方式下,任一寄存器移位的概率为 $3/4$ 。走遍一个循环周期大约需要 $(2^{23} - 1) \times 4/3$ 个时钟。

攻击 A5 要用 2^{40} 次加密来确定两个寄存器的结构,而后从密钥流决定第 3 个 LFSR。搜索密钥机已在设计之中 [Chambers 1994]。

A5 的基本想法不错,效率高,可通过所有已知统计检验标准。其唯一缺点是移存器级数短,其最短循环长度为 $4/3 \times 2^k$, k 是最长的 LFSR 的级数,总级数为 $19+22+23=64$ 。

可以用穷尽搜索法破译。若 A5 采用长的、抽头多的 LFSR，它会更安全。

4.4.2 加法流密码生成器

1. 加法生成器

以 nb 字为基本单元，其初始存数为 m 个 nb 字 $x_1, x_2, \dots, x_m$ 组成的阵列，按递归关系式给出 i 时刻的输出字 $x_i = a_{n-1}x_{i-1} + a_{n-2}x_{i-2} + \dots + a_1x_{i-n} + a_0x_{i-m} \bmod M$ 。其中， $+$ 号是 $\bmod M$ 加法运算，一般 $M = 2^m$ 。适当选择系数 $a_j (j = 0, 1, \dots, n-1)$ ，可使生成序列的周期极大化。Brent 给出了产生最大周期序列的条件。选用次数大于 2 的本原 3 次式，且由 Fibonacci 序列的最低位构成的数序列是以特征多项式 $x^n + \sum a'_i x^i$ ， $a'_i \equiv a_i \bmod 2$ 的 LFSR 所生成的序列。

例如，[55, 24, 0]所给定的递推式为

$$x_i = (x_{i-55} + x_{i-24}) \bmod 2^n$$

本原式中多于 3 项时，还需附加一些条件才能使周期为最大。称上述生成器为加法 (additive) 生成器。Knuth 曾以 Fibonacci 数决定递推式的系数，称其为滞后 (lagged) Fibonacci 生成器。由于这种生成器以字而不是按位生成密钥流，因此速度较快。

2. FISH 算法

Blöcher 等[1994]利用滞后 Fibonacci 生成器代替二元收缩式生成器，并增加一个映射 $f: GF(2^n) \rightarrow GF(2)$ 来生成 32b 的流密码和相应明文或密文异或实现加密和解密，称为 Fibonacci 收缩生成器，简称 FISH 算法。实现框图如图 4-7 所示。

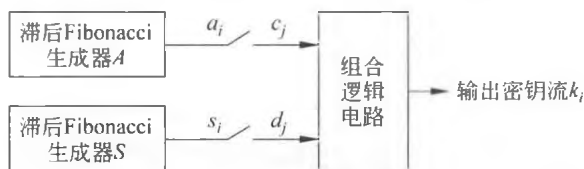


图 4-7 FISH 生成器

选 $n_A = 32, n_S = 32$ ， A 和 S 均为滞后 Fibonacci 生成器寄存器，其初始状态由密钥决定。滞后 Fibonacci 生成器的最低位的序列由一个本原 3 次多项式所决定的 LFSR 生成，满足

$$a_i = a_{i-55} + a_{i-24} \bmod 2^{32} \quad (4-12)$$

$$s_i = s_{i-52} + s_{i-19} \bmod 2^{32} \quad (4-13)$$

映射 $f: GF(2^{32}) \rightarrow GF(2)$ ，即将 S 寄存器的 32b 矢量映射为其最低位

$$f(b_{31}, b_{30}, \dots, b_0) = b_0 \quad (4-14)$$

若 $b_0 = 1$ ，则输出 a_i 和 s_i ，若 $b_0 = 0$ ，则丢弃 a_i 和 s_i ，继续移位运行。由此可以得到 32b 字序列 $c_0, c_1, \dots$ 和 $d_0, d_1, \dots$ ，将它们分别组对为 (c_{2i}, c_{2i+1}) 和 (d_{2i}, d_{2i+1}) ，并通过下述逻辑式得到

$$e_{2i} = c_{2i} \oplus (d_{2i} \wedge d_{2i+1}) \quad (4-15)$$

$$f_{2i} = d_{2i+1} \wedge (e_{2i} \wedge c_{2i+1}) \quad (4-16)$$

$$k_{2i} = e_{2i} \oplus f_{2i} \quad (4-17)$$

$$k_{2i+1} = c_{2i+1} \oplus f_{2i} \quad (4-18)$$

其中, $\oplus$ 表示逐位异或, $\wedge$ 表示逐位逻辑与。在 33MHz 的 PC 上可实现 15Mb/s 加密。已通过碰撞、相关、式样采集 (Coupon Collect)、频度、非线性复杂度、扑克、秩、串长、谱、重叠 m -重 (overlapping)、Ziv-Lempel 复杂度等检验, 表明它具有良好的随机性, 且特别适于软件快速实现。

3. PIKE 算法

虽然 FISH 通过了各类统计随机性检验, 但 Anderson 指出它仍不够安全。大约可用 2^{40} 次试验攻破。为此 Anderson 参照 A5 的设计思想, 对 FISH 进行改进, 提出所谓 PIKE 的算法。它采用 3 个 Fibonacci 生成器:

$$a_i = a_{i-55} + a_{i-24} \bmod 2^{32}$$

$$a_i = a_{i-57} + a_{i-7} \bmod 2^{32}$$

$$a_i = a_{i-58} + a_{i-19} \bmod 2^{32}$$

FISH 的控制位不是进位位, 而是最低位的位, 否则攻击会更难。因此 PIKE 采用进位位来控制。若所有 3 个进位位取值一样, 则 3 个寄存器都推进一位, 否则将推进两个有相同进位位的寄存器。控制将迟后 8 个循环, 每当更新状态之后, 就检查控制位, 并将一个控制 nybble 写到一个寄存器中。此寄存器以下一次更新存数移 4 位。在某些处理器下, 利用校验位作为控制可能更方便, 看来这是一种可接受的变通方法。

下一个密钥流字与 3 个寄存器的所有低位字进行异或。此算法较 FISH 稍快, 每个密钥流字平均需要 2.75 次更新计算值, 而不是 3 次。为了保证采用最小长度序列的比率很小, 限定在生成 2^{32} 个字后, 生成器重新注入密钥。缺少密钥供应的用户可以利用杂凑函数如 SHA 来扩充, 以提供 700 B 初始状态。此方案还没有经受多少密码分析。

4. Mush 算法

Mush 算法由 Wheeler 提出 [Schneier 1996], 采用两个 Fibonacci 生成器 A 和 B 进行相互钟控。若 A 有进位, 则 B 被驱动, 若 B 有进位, 则 A 被驱动。若 A 被驱动有进位时, 则置进位 bit; 若 B 被驱动有进位时, 则置进位 bit。最后输出密钥字由 A 和 B 的输出异或得到, 产生一个密钥字。平均需要 3 次迭代, 若适当选择系数, 且 A 与 B 的级数互素, 则可保证输出密钥流的周期极大化。目前尚无有关 Mush 的密码分析结果。

4.4.3 RC4

RC4 是由 RSA 安全公司的 Rivest 在 1987 年提出的密钥长度可变流密码, 但其算法细节一直未公开。1994 年 9 月有人在 Cypherpunks 邮递表中公布了 RC4 的源代码, 并通过 Internet 的 Usenet newsgroup sci.crypt 迅速传遍全球。虽然 RC4 已不能作为产品推销, 但 RSA 公司至今尚未公开有关它的文件 [Rivest 1992; Schneier 1996]。

该算法工作于 OFB 模式, 密钥流与明文独立, 利用 8×8 个 S 盒: $S_0, S_1, \dots, S_{255}$, 在变长密钥控制下对 0, 1, $\dots$, 255 的数进行置换。它有两个计数器 i 和 j , 初始时都为 0。

它通过下述算法产生随机字节:

$$i = (i + 1) \bmod 256$$

$$j = (j + S_i) \bmod 256$$

interchange S_i and S_j

$$t = (S_i + S_j) \bmod 256$$

$$K = S_t$$

字节 K 与明文异或得到密文，或与密文异或得到明文，其加密速度比 DES 快 10 倍。

S 盒的初始化过程如下：首先将其进行线性填数，即 $S_0=0, S_1=1, \dots, S_{255}=255$ ，然后以密钥填入另一个 256 字节的阵列，密钥不够长时可重复利用给定密钥以填满整个阵： $k_0, k_1, \dots, k_{255}$ 。将指数 j 置 0，并执行下述程序：

```
for i = 0 to 255
  j = (j + S_i + k_i) mod 256
  interchange S_i and S_j
```

RSA DSI 声称，RC4 对差分攻击和线性分析具有免疫力，没有短循环，且具有高度非线性。目前尚无它的公开分析结果。它大约有 $256! \times 256^2 = 2^{1700}$ 个可能的状态。各 S 在 i 和 j 的控制下卷入加密。指标 i 保证每个元素变化，指标 j 保证元素的随机改变。该算法简单明了，易于编程实现。

可以设想利用更大的 S 盒和更长的字，当然不一定要采用 16×16 个 S 盒，否则，初始化工作将极其漫长。

40b 密钥的 RC4 允许出口，但其安全性是无保证的。已有几十种采用 RC4 算法的商业产品，其中包括 Lottus Notes，Apple 公司的 AOEC，以及 Oracle Secure SQL，它也是美国移动通信技术公司的 CDPD 系统的一个组成部分。

关于分析 RC4 的攻击方法有许多公开发表的文献[Knudsen 等 1998; Mister 等 1998; Mantin 等 2001]，但没有哪种方法对于攻击足够长度的密钥（如 128 位）的 RC4 有效。值得注意的是，Fluhrer 等的报告指出，用于为 802.11 无线局域网提供机密性的 WEP，易于受到一种特殊攻击方法的攻击（见第 11 章）。从本质上讲，这个问题并不在 RC4 本身，而是作为 RC4 中输入密钥的生成途径有漏洞。这种特殊的攻击方式不适用于其他使用 RC4 的应用。通过修改 WEP 中密钥的生成途径，也可以避免这个攻击。这个问题恰恰说明设计一个安全系统的困难性不仅包括密码算法本身，还包括协议如何正确地使用这些密码算法。

4.4.4 祖冲之密码

2011 年 9 月 19—21 日，在日本福岡召开的第 53 次第三代合作伙伴计划（3GPP）系统架构组（SA）会议上，我国设计的祖冲之密码算法（ZUC）被批准成为新一代宽带无线移动通信系统（LTE）国际标准，即 4G 的国际标准。这是我国商用密码算法首次走出国门参与国际标准竞争，并取得重大突破。ZUC 成为国际标准提高了我国在移动通信领域的地位和影响力，对我国移动通信产业和商用密码产业发展均具有重要意义。

2012 年 3 月 21 日，国家密码管理局发布正式公告，将 ZUC 作为中国商用密码算法。

我国向 3GPP 提交的算法标准包含如下内容：

- (1) 祖冲之密码算法（ZUC）：用于产生密钥序列。
- (2) 128-EEA3：基于 ZUC 的机密性算法。
- (3) 128-EIA3：基于 ZUC 的完整性保护算法。

1. ZUC 算法

ZUC 本质上是一个密钥序列产生算法，其输入为 128 比特的初始密钥和 128 比特的初始向量，输出为 32 比特的密钥字序列。其逻辑上分为三层，分别是：16 级线性反馈移位寄存器（LFSR），比特重组（BR），非线性函数 F 。

① LFSR 以一个有限域 $GF(2^{31}-1)$ 上的 16 次本原多项式为连接多项式，输出为 $GF(2^{31}-1)$ 上的 m 序列。

② BR 从 LFSR 的状态中取出 128 位，拼成 4 个 32 位字 (x_0, x_1, x_2, x_3)。非线性函数 F 从 BR 接受 3 个 32 位字 (x_0, x_1, x_2)，经过异或、循环移位、模 2^{32} 、非线性 S 盒变换，输出 32 位字 W 。

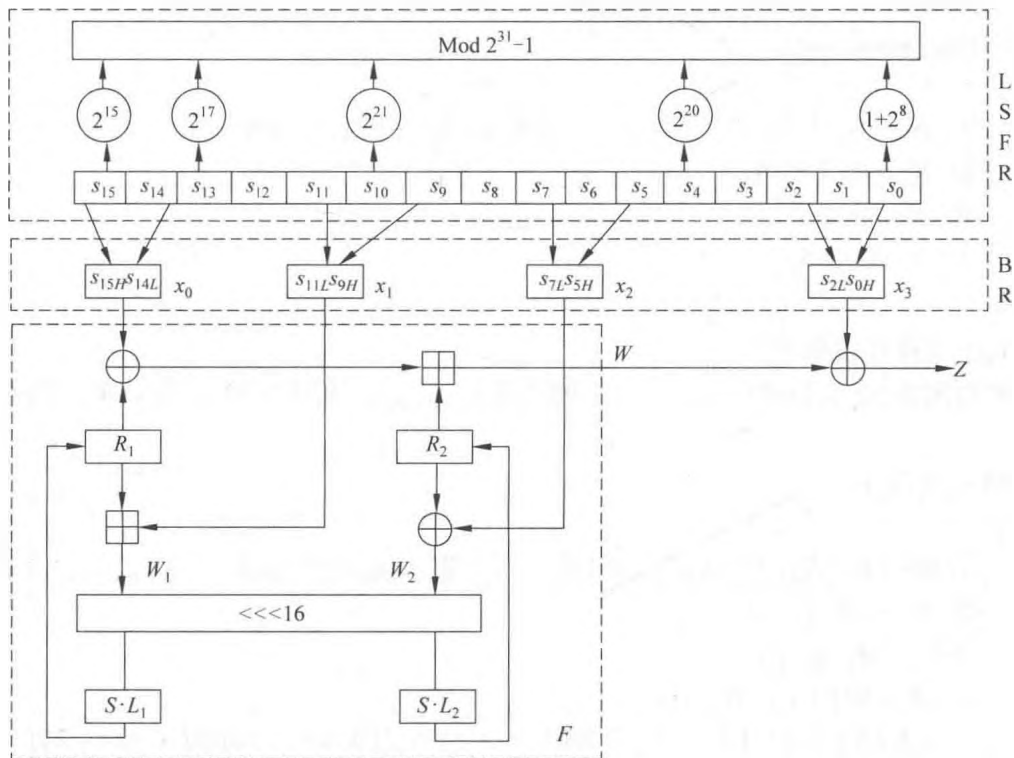


图 4-8 ZUC 算法结构图

(1) 线性反馈移位寄存器（LFSR）

LFSR 由 16 个 31 位的寄存器 ($s_0, s_1, \dots, s_{14}, s_{15}$) 组成，每一个都是定义在素域 $GF(2^{31}-1)$ 上。LFSR 有两种状态：初始化状态和工作状态。详细步骤如下所述。

LFSRWithInitialisationMode(u)

{

- ① $v = 2^{15}s_{15} + 2^{17}s_{13} + 2^{21}s_{10} + 2^{20}s_4 + (1+2^8)s_0 \bmod (2^{31}-1)$;
- ② $s_{16} = (v+u) \bmod (2^{31}-1)$; // u 是 w 通过舍弃最低位比特得到
- ③ If $s_{16}=0$, then set $s_{16} = 2^{31}-1$;
- ④ $(s_1, s_2, \dots, s_{15}, s_{16}) \rightarrow (s_0, s_1, \dots, s_{14}, s_{15})$ 。

}

LFSRWithWorkMode()

{

- ① $s_{16} = 2^{15}s_{15} + 2^{17}s_{13} + 2^{21}s_{10} + 2^{20}s_4 + (1+2^8)s_0 \bmod (2^{31}-1)$;
- ② If $s_{16}=0$, then set $s_{16} = 2^{31}-1$;
- ③ $(s_1, s_2, \dots, s_{15}, s_{16}) \rightarrow (s_0, s_1, \dots, s_{14}, s_{15})$ 。

}

(2) 比特重组 (BR)

比特重组是一个过渡层，其主要从 LFSR 的 8 个寄存器单元抽取 128 比特内容组成 4 个 32 比特的字，以供下层非线性函数 F 和密钥输出使用。详细步骤为

Bitreorganization()

{

- ① $X_0 = s_{15H} \parallel s_{14L}$; //其中符号 $\parallel$ 表示两个字符首尾拼接
- ② $X_1 = s_{11L} \parallel s_{9H}$;
- ③ $X_2 = s_{7L} \parallel s_{5H}$;
- ④ $X_3 = s_{2L} \parallel s_{0H}$ 。

}

(3) 非线性函数 F

F 有两个 32 位存储单元 R_1 、 R_2 ，输入为 x_0, x_1, x_2 ，输出为 32 位的字 W 。详细步骤为

$F(x_0, x_1, x_2)$

{

- ① $W = (X_0 \oplus R_1) \boxed{+} R_2$; //其中符号 $\boxed{+}$ 表示 $\bmod 2^{32}$ 加法
- ② $W_1 = R_1 \boxed{+} x_1$;
- ③ $W_2 = R_2 \oplus x_2$;
- ④ $R_1 = S(L_1(W_{1L} \parallel W_{2H}))$;
// $L_1(X) = X \oplus (X \lll_{32} 2) \oplus (X \lll_{32} 10) \oplus (X \lll_{32} 18) \oplus (X \lll_{32} 24)$
- ⑤ $R_2 = S(L_2(W_{2L} \parallel W_{1H}))$;
// $L_2(X) = X \oplus (X \lll_{32} 8) \oplus (X \lll_{32} 14) \oplus (X \lll_{32} 22) \oplus (X \lll_{32} 30)$
//下标32表示 X 是32位的数; S 为S盒运算。

}

这里的S盒由4个并置的8进8出的S盒构成，即 $S = (S_0, S_1, S_2, S_3)$ ，其中 $S_2 = S_0$ ， $S_3 = S_1$ ，于是有 $S = (S_0, S_1, S_0, S_1)$ 。S盒 S_0 和 S_1 的置换运算如表4-1和表4-2所示。

表 4-1 S 盒 S_0

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	3E	72	5B	47	CA	E0	00	33	04	D1	54	98	09	B9	6D	CB
1	7B	1B	F9	32	AF	9D	6A	A5	B8	2D	FC	1D	08	53	03	90
2	4D	4E	84	99	E4	CE	D9	91	DD	B6	85	48	8B	29	6E	AC
3	CD	C1	F8	1E	73	43	69	C6	B5	BD	FD	39	63	20	D4	38
4	76	7D	B2	A7	CF	ED	57	C5	F3	2C	BB	14	21	06	55	9B
5	E3	EF	5E	31	4F	7F	5A	A4	0D	82	51	49	5F	BA	58	1C
6	4A	16	D5	17	A8	92	24	1F	8C	FF	D8	AE	2E	01	D3	AD
7	3B	4B	DA	46	EB	C9	DE	9A	8F	87	D7	3A	80	6F	2F	C8
8	B1	B4	37	F7	0A	22	13	28	7C	CC	3C	89	C7	C3	96	56
9	07	BF	7E	F0	0B	2B	97	52	35	41	79	61	A6	4C	10	FE
A	BC	26	95	88	8A	B0	A3	FB	C0	18	94	F2	E1	E5	E9	5D
B	D0	DC	11	66	64	5C	EC	59	42	75	12	F5	74	9C	AA	23
C	0E	86	AB	BE	2A	02	E7	67	E6	44	A2	6C	C2	93	9F	F1
D	F6	FA	36	D2	50	68	9E	62	71	15	3D	D6	40	C4	E2	0F
E	8E	83	77	6B	25	05	3F	0C	30	EA	70	B7	A1	E8	A9	65
F	8D	27	1A	DB	81	B3	A0	F4	45	7A	19	DF	EE	78	34	60

表 4-2 S 盒 S_1

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	55	C2	63	71	3B	C8	47	86	9F	3C	DA	5B	29	AA	FD	77
1	8C	C5	94	0C	A6	1A	13	00	E3	A8	16	72	40	F9	F8	42
2	44	26	68	96	81	D9	45	3E	10	76	C6	A7	8B	39	43	E1
3	3A	B5	56	2A	C0	6D	B3	05	22	66	BF	DC	0B	FA	62	48
4	DD	20	11	06	36	C9	C1	CF	F6	27	52	BB	69	F5	D4	87
5	7F	84	4C	D2	9C	57	A4	BC	4F	9A	DF	FE	D6	8D	7A	EB
6	2B	53	D8	5C	A1	14	17	FB	23	D5	7D	30	67	73	08	09
7	EE	B7	70	3F	61	B2	19	8E	4E	E5	4B	93	8F	5D	DB	A9
8	AD	F1	AE	2E	CB	0D	FC	F4	2D	46	6E	1D	97	E8	D1	E9
9	4D	37	A5	75	5E	83	9E	AB	82	9D	B9	1C	E0	CD	49	89
A	01	B6	BD	58	24	A2	5F	38	78	99	15	90	50	B8	95	E4
B	D0	91	C7	CE	ED	0F	B4	6F	A0	CC	F0	02	4A	79	C3	DE
C	A3	EF	EA	51	E6	6B	18	EC	1B	2C	80	F7	74	E7	FF	21
D	5A	6A	54	1E	41	31	92	35	C4	33	07	0A	BA	7E	0E	34
E	88	B1	98	7C	F3	3D	60	6C	7B	CA	D3	1F	32	65	04	28
F	64	BE	85	9B	2F	59	8A	D7	B0	25	AC	AF	12	03	E2	F2

(4) 密钥封装

密钥封装过程将 128 位的初始密钥 KEY 和 128 位的初始向量 IV 扩展为 16 个 31 位字作为 LFSR 变量 $s_0, s_1, \dots, s_{14}, s_{15}$ 的初始状态。设 KEY 和 IV 分别为：

$$\text{KEY} = k_0 \parallel k_1 \parallel k_2 \parallel \dots \parallel k_{15}$$

$$\text{IV} = iv_0 \parallel iv_1 \parallel iv_2 \parallel \dots \parallel iv_{15}$$

则密钥封装过程如下：

① 设 D 为 240 位常量，按如下方式分成 16 个 15 位的字串： $D = d_0 \parallel d_1 \parallel \dots \parallel d_{15}$ ；

② 对于 $0 \leq i \leq 15$ ，有 $s_i = k_i \parallel d_i \parallel iv_i$ 。

(5) 算法运行

ZUC 算法运行，分为初始化阶段和工作阶段。

初始化阶段将 128 位的初始密钥 K 和 128 位的初始向量 IV 按照上面的密钥封装方法封装到 LFSR 的寄存器单元变量 $s_0, s_1, \dots, s_{14}, s_{15}$ 中，作为 LFSR 的初态， R_1 、 R_2 也初始化为 0，重复执行下述过程 32 次：

① Bitreorganization()；

② $W = F(x_0, x_1, x_2)$ ；

③ LFSRWithInitialisationMode($u \gg 1$)。

工作阶段首先需要先将下面的操作运行一轮：

① Bitreorganization()；

② $F(x_0, x_1, x_2)$ ； //此处丢弃输出结果

③ LFSRWithWorkMode()。

然后进入密钥输出阶段，将下面的操作运行一次就会生成一个 32 比特密钥 Z 。

① Bitreorganization()；

② $Z = F(x_0, x_1, x_2) \oplus x_3$ ；

③ LFSRWithWorkMode()。

2. 基于 ZUC 的机密性算法 128-EEA3

128-EEA3 主要用于 4G 移动通信中移动用户设备 (User Equipment, UE) 和核心网 (Core Network) 之间无线链路上信令和数据的加解密。128-EEA3 加解密原理如图 4-9 所示。

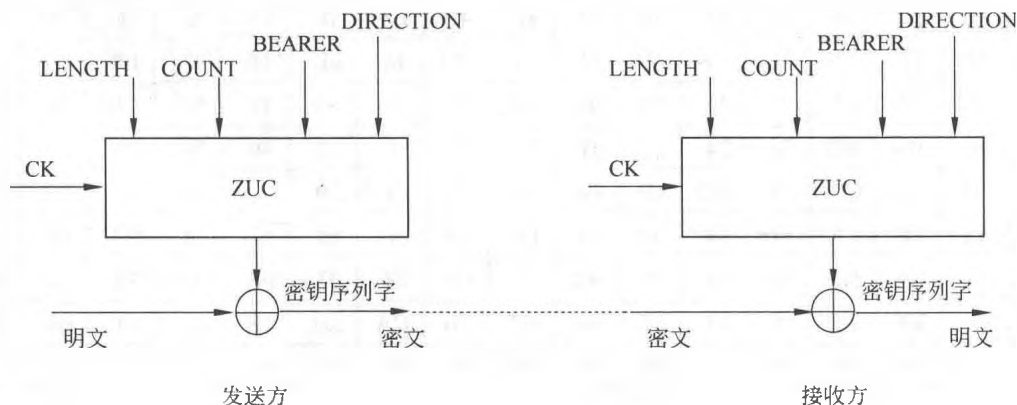


图 4-9 128-EEA3 算法原理图

利用初始密钥 KEY 和初始向量 IV, 执行 ZUC 算法, 产生 L 个 32 位字的加解密密钥流。设长度为 LENGTH 的输入比特流为:

$$\text{IBS} = \text{IBS}[0] \parallel \text{IBS}[1] \parallel \cdots \parallel \text{IBS}[\text{LENGTH}-1]$$

对应的输出比特流为: $\text{OBS} = \text{OBS}[0] \parallel \text{OBS}[1] \parallel \cdots \parallel \text{OBS}[\text{LENGTH}-1]$

加解密只需要把明文(密文)与加解密密钥模 2 相加即可:

$$\text{OBS}[i] = \text{IBS}[i] \oplus K[i], \quad i=0, 1, 2, \cdots, \text{LENGTH}-1$$

输入参数定义如下:

LENGTH: 明文消息流的比特长度, 32 位

COUNT: 计数器, 32 位

BEARER: 承载层标识, 5 位

DIRECTION: 传输方向标识, 1 位

CK: 机密性密钥, 128 位, 由 ZUC 产生

3. 基于 ZUC 的完整性算法 128-EIA3

128-EIA3 主要用于 4G 移动通信中移动 UE 和核心网之间的无线链路上的通信信令和数据的完整性认证, 并对信令源进行认证。主要由 128-EIA3 产生消息认证码(MAC), 通过验证 MAC 值, 实现对消息的完整性认证。

128-EIA3 的工作原理如图 4-10 所示。

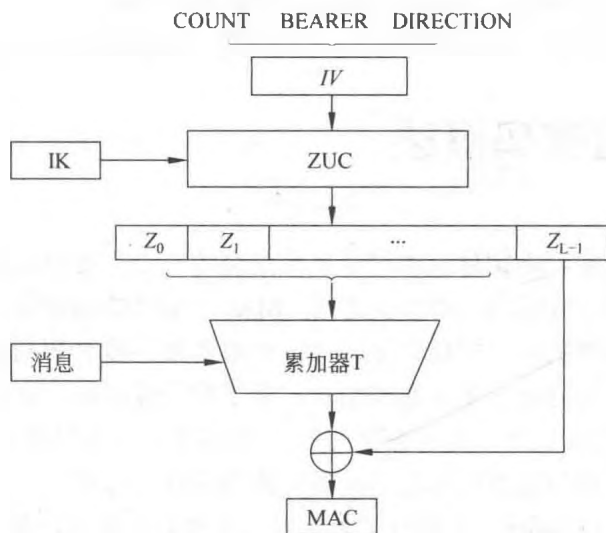


图 4-10 128-EIA3 算法原理图

利用初始密钥 KEY 和初始向量 IV, 执行 ZUC 算法, 产生 L 个 32 位的完整性密钥字流。设需要计算消息认证码的消息比特序列为

$$M = m[0], m[1], \cdots, m[\text{LENGTH}-1].$$

设 T 为一个 32 比特的字变量, MAC 计算如下:

MACComputation()

{

```

① Set  $T = 0$ ;
② For ( $I = 0$ ;  $I < \text{LENGTH}$ ;  $I++$ )
    If  $m[I] = 1$  then  $T = T \oplus K_i$ ;      //  $K_i = k[i] \parallel k[i+1] \parallel \dots \parallel k[i+31]$ 
③ End For
④  $T = T \oplus K_{\text{LENGTH}}$ ;
⑤  $\text{MAC} = T \oplus K_{32 \times (\text{L}-1)}$ 
}

```

最后，讨论一下 ZUC 算法的安全性。

ZUC 算法在 LFSR 层采用了 $\text{GF}(2^{31}-1)$ 上的 16 次本原多项式，其输出序列随机性好、周期足够大。在比特重组部分，重组的数据具有良好的随机性，且出现的重复概率足够小。在非线形函数 F 中采用了两个存储部件 R 、二个线性部件 L 和两个非线性 S 盒，使其输出具有良好的非线性、混淆特性和扩散特性。设计者经过评估，认为能够抵抗弱密钥攻击、Guess-and-Determine 攻击、Binary Decision trees 攻击、线性区分攻击、代数攻击和选择初始向量攻击等多种密码攻击。

在侧信道攻击方面，理论分析与实验表明，ZUC 算法经不起 DPA 类侧信道的攻击。因此在硬件实现时必须采取保护措施。

另外，128-EIA3 长度为 32 位，穷举攻击的复杂度为 $O(2^{32})$ ，显然太短了。这可能是移动通信的实时性要求导致。实际应用中应当采取保护措施。

随着使用时间的推移，ZUC 算法安全性的理论分析和实践检验会更加充分。

4.5 分组密码概述

在许多密码系统中，单钥分组密码是系统安全的一个重要组成部分。分组密码易于构造拟随机数生成器、流密码、消息认证码（MAC）和杂凑函数等，还可进而成为消息认证技术、数据完整性机构、实体认证协议以及单钥数字签名体制的核心组成部分。实际应用中对于分组码可能提出多方面的要求，除了安全性以外，还有运行速度、存储量（程序的长度、数据分组长度、高速缓存大小）、实现平台（软硬件、芯片）、运行模式等限制条件。这些都需要与安全性要求之间进行适当的折中选择。

分组密码（Block Cipher）是将明文消息编码表示后的数字序列 $x_1, x_2, \dots, x_t$ ，划分成长为 m 的组 $\mathbf{x} = (x_0, x_1, \dots, x_{m-1})$ ，各组（长为 m 的矢量）分别在密钥 $k = (k_0, k_1, \dots, k_{t-1})$ 控制下变换成等长的输出数字序列 $\mathbf{y} = (y_0, y_1, \dots, y_{n-1})$ （长为 n 的矢量），其加密函数 $E: V_m \times K \rightarrow V_n$ ， $V_m (V_n)$ 是 $m(n)$ 维矢量空间， K 为密钥空间，如图 4-11 所示。它与流密码的不同之处在于输出的每一位数字不是只与相应时刻输入的明文数字有关，而是与一组长为 m 的明文数字有关。在相同密钥下，分组密码对长为 m 的输入明文组所实施的变换是等同的，所以只需研究对任一组明文数字的变换规则。这种密码实质上是字长为 m 的数字序列的代换密码。

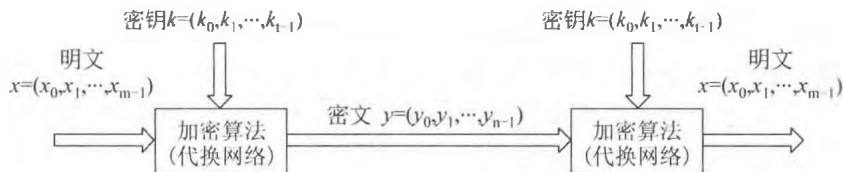


图 4-11 分组密码框图

通常取 $n=m$ 。若 $n>m$ ，则为有数据扩展的分组密码。若 $n<m$ ，则为有数据压缩的分组密码。在二元情况下， x 和 y 均为二元数字序列，它们的每个分量 $x_i, y_i \in \text{GF}(2)$ 。下面主要讨论二元情况。将长为 n 的二元 x 和 y 表示成小于 2^n 的整数，即

$$x = (x_0, x_1, \dots, x_{n-1}) \leftrightarrow \sum_{i=0}^{n-1} x_i 2^i = \|x\| \quad (4-19)$$

$$y = (y_0, y_1, \dots, y_{n-1}) \leftrightarrow \sum_{i=0}^{n-1} y_i 2^i = \|y\| \quad (4-20)$$

则分组密码就是将 $\|x\| \in \{0, 1, \dots, 2^n - 1\}$ 映射为 $\|y\| \in \{0, 1, \dots, 2^n - 1\}$ ，即为 $\{0, 1, \dots, 2^n - 1\}$ 到其自身的一个置换 π ，即

$$y = \pi(x) \quad (4-21)$$

置换的选择由密钥 k 决定。所有可能置换构成一个对称群 $\text{SYM}(2^n)$ ，其中元素个数或密钥数为

$$\# \{\pi\} = 2^n! \quad (4-22)$$

例如 $n=64b$ 时，

$$(2^{64})! > 10^{347\ 380\ 000\ 000\ 000\ 000\ 000} > (10^{10})^{20}$$

为表示任一特定置换所需的二元数字位数为

$$\log_2(2^n!) \approx (n-1.44)2^n = o(n2^n)b \quad (4-23)$$

即密钥长度达 $n2^n b$ ， $n=64$ 时的值为 $64 \times 2^{64} = 2^{70}b$ ，DES 的密钥仅为 $56b$ ，IDEA 的密钥也不过为 $128b$ 。实用中的各种分组密码（如后面要介绍的 DES、IDEA、RSA 和背包体制等）所用的置换都不过是上述置换集中的一个很小的子集。分组密码的设计问题在于找到一种算法，能在密钥控制下从一个足够大且足够好的置换子集中，简单而迅速地选出一个置换，用来对当前输入的明文的数字组进行加密变换。因此，设计的算法应满足下述要求：

(1) 分组长度 n 要足够大，使分组代换字母表中的元素个数 2^n 足够大，防止明文穷举攻击奏效。DES、IDEA、FEAL 和 LOKI 等分组密码都采用 $n=64$ ，在生日攻击下用 2^{32} 组密文成功概率为 $1/2$ ，同时要求 $2^{32} \times 64b = 2^{15} \text{ MB}$ 存储空间，故采用穷举攻击是不现实的。

(2) 密钥量要足够大（即置换子集中的元素足够多），尽可能消除弱密钥并使所有密钥同等，以防止密钥穷举攻击奏效。但密钥又不能过长，以利于密钥的管理。DES 采用 $56b$ 密钥，看来太短了，IDEA 采用 $128b$ 密钥，Denning 等估计，在今后 $30 \sim 40$ 年内采用 $80b$ 密钥是足够安全的。

(3) 由密钥确定置换的算法要足够复杂，充分实现明文与密钥的扩散和混淆，没有

简单的关系可循，要能抗击各种已知的攻击，如差分攻击和线性攻击等；有高的非线性阶数，实现复杂的密码变换，使对手在破译时除了用穷举法外，无其他捷径可循。

应当指出，上述有关安全性条件都是必要条件，是设计分组密码时应当充分考虑的一些问题，但绝不是安全性的充分条件。

(4) 加密和解密运算简单，易于软件和硬件高速实现。如将分组 n 划分为子段，每段长为 8、16 或者 32。在以软件实现时，应选用简单的运算，使作用于子段上的密码运算易于以标准处理器的基本运算，如加、乘、移位等实现，避免用以软件难以实现的逐位置换。为了便于硬件实现，加密和解密过程之间的差别应仅在于由秘密密钥所生成的密钥表不同。这样，加密和解密就可用同一器件实现。设计的算法采用规则的模块结构，如多轮迭代等，以便于采用软件和 VLSI 快速实现。

(5) 数据扩展。一般无数据扩展，在采用同态置换和随机化加密技术时可引入数据扩展。

(6) 差错传播尽可能得小。

要实现上述几点要求并不容易。首先，图 4-11 的代换网络的复杂性随分组长度 n 呈指数增大，常常会使设计变得复杂而难以控制和实现；实际中常常将 n 分成几个小段，分别设计各段的代换逻辑实现电路，采用并行操作达到总的分组长度 n 足够大，这将在下面讨论。其次，为了便于实现，实际中常常将较简单易于实现的密码系统进行组合，构成较复杂的、密钥量较大的密码系统。Shannon[1949]曾提出了以下两种可能的组合方法。

(1) “概率加权和”方法，即以一定的概率随机地从几个子系统中选择一个用于加密当前的明文。设有 r 个子系统，以 $T_1, T_2, \dots, T_r$ 表示，相应被选用的概率为 $p_1, p_2, \dots, p_r$ ，

其中 $\sum_{i=1}^r p_i = 1$ 。其概率和系统可表示成

$$T = p_1 T_1 + p_2 T_2 + \dots + p_r T_r \quad (4-24)$$

显然，系统 T 的密钥量将是各子系统密钥量之和。

(2) “乘积”方法。例如，设有两个子密码系统 T_1 和 T_2 ，则先以 T_1 对明文进行加密，然后再以 T_2 对所得结果进行加密。其中， T_1 的密文空间需作为 T_2 的“明文”空间。乘积密码可表示成

$$T = T_1 T_2 \quad (4-25)$$

利用这两种方法可将简单易于实现的密码组合成复杂的更为安全的密码。

最后，为了抗击统计分析破译法，需要实现第 3 条要求，Shannon 曾建议采用扩散 (diffusion) 和混淆 (confusion) 法。所谓扩散，就是将每位明文及密钥数字的影响尽可能迅速地散布到较多个输出的密文数字中，以便隐蔽明文数字的统计特性。这一想法可推广到将任一位密钥数字的影响尽量迅速地扩展到更多个密文数字中去，以防止对密钥进行逐段破译。在理想情况下，明文的每位和密钥的每位应影响密文的每位，即实现所谓“完备性”。Shannon 提出的“混淆”概念目的在于使作用于明文的密钥和密文之间的关系复杂化，使明文和密文之间、密文和密钥之间的统计相关性极小化，从而使统计分析攻击法不能奏效。他用“揉面团”过程来形象地比喻“扩散”和“混淆”概念。在设

计实际密码算法时,需要巧妙地运用这两个概念。与揉面团不同,将明文和密钥进行“混合”作用时还需满足两个条件:一是变换必须是可逆的,并非任何混淆办法都能做到这点;二是变换和反变换过程应当简单易行。乘积密码有助于实现扩散和混淆,选择某个较简单的密码变换,在密钥控制下以迭代方式多次利用它进行加密变换,就可实现预期的扩散和混淆效果。当代提出的各种分组密码算法,都在一定程度上体现了 Shannon 构造密码的这些重要思想。

4.6 数据加密标准

数据加密标准(DES)中的算法是第一个并且也是十分重要的现代对称加密算法。1977年1月,美国国家标准局公布了 DES,它是用于非保密数据(与国家安全无关的信息)的算法,该算法在世界范围内已经得到了广泛的应用,一个主要的例子就是银行用它保护资金转账安全。本来该标准被批准使用5年,但由于它经受住了时间的考验,随后又批准了3个5年的使用期。

4.6.1 DES 介绍

DES 是分组密码,其中的消息被分成定长的数据分组,每一分组称为 M 或 C 中的一个消息。在 DES 中,有 $M = C = \{0,1\}^{64}$, $K = \{0,1\}^{56}$,也就是 DES 加密和解密算法输入 64b 明文或密文消息和 56b 密钥,输出 64b 密文或明文消息。

DES 的运算可描述为如下 3 步:

(1) 对输入分组进行固定的“初始置换”IP,可以将这个初始置换写为

$$(L_0, R_0) \leftarrow \text{IP}(\text{Input Block}) \quad (4-26)$$

这里 L_0 和 R_0 称为“(左,右)半分组”,都是 32b 的分组。注意,IP 是固定的函数(也就是说,输入密钥不是它的参数),是公开的,因此这个初始置换在密码学上意义不大。

(2) 将下面的运算迭代 16 轮($i=1,2,\dots,16$)

$$L_i \leftarrow R_{i-1} \quad (4-27)$$

$$R_i \leftarrow L_{i-1} \oplus f(R_{i-1}, k_i) \quad (4-28)$$

这里 k_i 称为“轮密钥”,它是 56b 输入密钥的一个 48b 的子串, f 称为“S 盒函数”(“S”表示代换,将在 4.6.2 节中对这个函数进行简单描述),是一个代换密码。这个运算的特点是交换两半分组,就是说,一轮的左半分组输入是上一轮的右半分组输出。交换运算是一个简单的换位密码(见 4.2.2 节),目的是获得很大程度的“信息扩散”,本质上就是获得式(4-26)中香农提出的模型的混合特性。从我们的讨论中可以看出,DES 的这一步是代换密码和换位密码的结合。

(3) 将 16 轮迭代后得到的结果 (L_{16}, R_{16}) 输入到 IP 的逆置换来消除初始置换的影响,这一步的输出就是 DES 算法的输出,我们将最后一步写为

$$\text{Output Block} \leftarrow \text{IP}^{-1}(R_{16}, L_{16}) \quad (4-29)$$

请特别注意 IP^{-1} 的输入：在输入 IP^{-1} 以前，16 轮迭代输出的两个半分组又进行了一次交换。

加密和解密算法都用这 3 个步骤，仅有的不同就是，如果加密算法中使用的轮密钥是 $k_1, k_2, \dots, k_{16}$ ，那么解密算法中使用的轮密钥就应当是 $k_{16}, k_{15}, \dots, k_1$ ，这种排列轮密钥的方法称为“密钥表”，可以记为

$$(k'_1, k'_2, \dots, k'_{16}) = (k_{16}, k_{15}, \dots, k_1) \quad (4-30)$$

例 4-4 在加密密钥 k 下，将明文消息 m 加密为密文消息 c ，下面通过 DES 算法来确认解密函数的正确运行，也就是在 k 下， c 的解密将输出 m 。

解密算法首先输入密文 c 作为“输入分组”。由式 (4-26) 有

$$(L'_0, R'_0) \leftarrow IP(c)$$

但是，因为 c 实际上是加密算法中最后一步的“输出分组”，由式 (4-29) 有

$$(L'_0, R'_0) \leftarrow (R_{16}, L_{16}) \quad (4-31)$$

在第 1 轮中，由式 (4-27)、式 (4-28) 和式 (4-30)，有

$$L'_1 \leftarrow R'_0 = L_{16}$$

$$R'_1 \leftarrow L'_0 \oplus f(R'_0, k'_1) = R_{16} \oplus f(L_{16}, k'_1)$$

在这两个式子的右边，由式 (4-27) 可知， L_{16} 应该用 R_{15} 代替；由式 (4-28) 可知， R_{16} 应该用 $L_{15} \oplus f(R_{15}, k_{16})$ 代替。根据密钥表式 (4-30)， $k'_1 = k_{16}$ ，因此，上面两个式子实际上是下面的两个：

$$L'_1 \leftarrow R_{15}$$

$$R'_1 \leftarrow [L_{15} \oplus f(R_{15}, k_{16})] \oplus f(R_{15}, k_{16}) = L_{15}$$

所以，在第 1 轮解密以后得到

$$(L'_1, R'_1) \leftarrow (R_{15}, L_{15})$$

因此，在第 2 轮开始，两个半分组是 (R_{15}, L_{15}) 。

在随后的 15 轮中，使用同样的验证，将获得

$$(L'_2, R'_2) \leftarrow (R_{14}, L_{14}), \dots, (L'_{16}, R'_{16}) \leftarrow (R_0, L_0)$$

从 16 轮迭代得到的两个最后的半分组 (L'_{16}, R'_{16}) 被交换为 $(R'_{16}, L'_{16}) = (L_0, R_0)$ ，然后输入到 IP^{-1} （注意式 (4-29) 中另外一次的交换）来消除 IP 在式 (4-26) 中的影响。解密函数的输出确实就是最初的明文分组 m 。

已经证明：DES 加密和解密算法确实使得方程式 (4-26) 对于所有的 $m \in M$ 和 $k \in K$ 都成立。很明显，这些算法的运行与“S 盒函数”的内部细节及密钥表函数无关。

使用式 (4-27) 和式 (4-28) 以交换的方式处理两个半分组的 DES 迭代称为 Feistel 密码。图 4-12 给出了一轮 Feistel 密码的交换结构。最初是由 Feistel 提出了这个密码。像以前提到的那样，交换特性的目的是为了获得一个较程度上的数据扩散。Feistel 密码在公钥密码学中也有重要的应用：称为最佳非对称加密填充

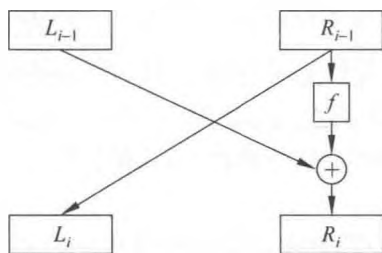


图 4-12 Feistel 密码（一轮）

(OAEP)。其在结构本质上是一个二轮的 Feistel 密码。

4.6.2 DES 的核心作用：消息的随机非线性分布

DES 的核心部分是在“S 盒函数” f 中。正是在这里 DES 实现了明文消息在密文消息空间上的随机非线性分布。

在第 i 轮, $f(R_{i-1}, k_i)$ 做下面的两个子运算:

(1) 通过逐比特异或运算, 将轮密钥 k_i 与半分组 R_{i-1} 相加。这提供了消息分布中所需要的随机性。

(2) 在包含 8 个“代换盒”(S 盒)的固定置换下代换 (i) 的结果, 每一个 S 盒是一个非线性置换函数; 这就提供了消息分布中所需的非线性。

S 盒的非线性对 DES 的安全是非常重要的, 注意到代换密码(例如, 有随机密钥的例 4-1)在一般情况下是非线性的, 而移位密码和仿射密码是线性中的子类。与一般情况相比, 这些线性子类不仅极大地减小了密钥空间, 而且也导致了生成的密文对于差分分析(DC)技术是脆弱的[Biham 等 1991]。DC 通过利用两个明文消息间的线性差分 and 两个密文消息间的线性差分来攻击密码, 下面以仿射密码式(4-6)为例分析这种攻击。假定 Malice(攻击者)以某种方式知道了差分 $m - m'$, 但他既不知道 m 也不知道 m' , 给定相应的密文 $c = k_1 m + k_2 (\text{mod } N)$, $c' = k_1 m' + k_2 (\text{mod } N)$, Malice 可以计算

$$k_1 = (c - c') / (m - m') (\text{mod } N)$$

有了 k_1 , Malice 进一步找到 k_2 就变得容易多了, 例如, 如果 Malice 有一个已知的明文-密文对, 他就能找到 k_2 。在 1990 年发现了 DC 以后, 对于许多已知的分组密码的攻击, DC 已经被证明是非常有效的, 然而它攻击 DES 并不是非常成功。这就表明 DES 的设计者早在 15 年前通过 S 盒的非线性设计就已采取了预防 DC 的措施。

DES(事实上还有 Feistel 密码)的一个有趣的特点就是函数 $f(R_{i-1}, k_i)$ 中的 S 盒不必是可逆的。在例 4-4 中对于任意的 $f(R_{i-1}, k_i)$ 都可运行加密和解密就证明了这一点, 这个特点节约了 DES 硬件实现的空间。

本书将省略对 S 盒的内部细节、密钥表函数和初始置换函数的描述, 这些细节超出了本书的范围, 有兴趣的读者可在文献[Denning 1982]中找到这些细节。

4.6.3 DES 的安全性

在 DES 作为加密标准提出之后不久, 学者们就开始争论 DES 的安全性。其详细的讨论和历史描述可以在各种密码学教科书中找到, 如文献[Smid 等 1992]、[Stinson 1995]和[Menezes 等 1997]。后来, 人们越来越清楚, 这些讨论找到了 DES 的一个主要的缺点: DES 的密钥长度较短。这被认为是 DES 仅有的最严重的弱点, 针对这个弱点的攻击包括穷举测试密钥, 就是利用一个已知的明文和密文消息对, 直到找到正确的密钥, 这就是所谓的强力或穷举密钥搜索攻击。

然而, 不能将强力密钥搜索攻击看做是一种真正的攻击, 这是因为密码设计者不仅已经预见到了它, 而且希望这是对手仅有的工具, 因此, 假设攻击者仅具有 20 世纪 70 年

代的计算技术,那么 DES 是一种十分成功的密码。

克服短密钥缺陷的一个解决办法是使用不同的密钥,多次运行 DES 算法,那样的一个方案称为加密-解密-加密 3 重 DES 方案[Tuchman 1979]。这个方案中的加密记为

$$c \leftarrow E_{k_1}(D_{k_2}(E_{k_1}(m)))$$

解密记为

$$m \leftarrow D_{k_1}(E_{k_2}(D_{k_1}(c)))$$

除了能够达到扩大密钥空间的效果,如果使用 $k_1 = k_2$,这个方案也很容易与单钥 DES 兼容。3 重 DES 也可以使用 3 个不同的密钥,但这时它与单钥 DES 不兼容。

DES 的短密钥弱点在 20 世纪 90 年代变得明显了。在 1993 年,Wiener 认为花费 1 000 000 美元可以造一个特殊用途的 VLSI DES 密钥搜索机,给定一个明文-密文消息对,预计这台机器将在 3.5 h 之内找到密钥。1998 年 7 月 15 日,密码学研究会、高级无线技术协会和电子前沿基金会(Electronic Frontier Foundation, EFF)联合宣布了破纪录的 DES 密钥搜索攻击:他们花了不到 250 000 美元构造了一个称为 DES 解密高手(也称做 Deep Crack)的密钥搜索机,搜索了 56 h 后成功地找到了 RSA 的 DES 挑战密钥。这个结果表明:对于一个安全的单钥密码来说,在 20 世纪 90 年代后期的计算技术背景下,使用 56b 的密钥太短了。

4.7

高级加密标准

1997 年 1 月 2 日,美国国家标准和技术协会(NIST)宣布征集一个新的对称密钥分组密码算法作为取代 DES 的新的加密标准。这个新的算法被命名为高级加密标准(AES)。与 DES 的封闭设计过程不同,在 1997 年 9 月 12 日,正式地公开征集 AES 算法,规定了 AES 要详细说明一个非保密的、公开的对称密钥加密算法(s);算法(s)必须支持(至少)128b 的分组长度,以及 128b、192b 和 256b 的密钥长度,强度应该相当于 3 重 DES,但是应该比 3 重 DES 更有效。此外,如果算法(s)被选中,在世界范围内它必须是可免费获得的。

1998 年 8 月 20 日,NIST 公布了 15 个 AES 候选算法,这些算法由遍布世界的密码团体的成员提交。公众对这 15 个算法的评论被当作这些算法的初始评论(公众的初始评论期也称为第 1 轮),第 1 轮评选到 1999 年 4 月 15 日截止。根据收到的分析和评论,NIST 从 15 个算法中选出 5 个算法,这 5 个参加决赛的候选算法是 MARS[Burwick 等 1998]、RC6[Sidney 等 1998]、Rijndael[Daemen 等 1998]、Serpent[Anderson 等 1998]和 Twofish[Schneier 等 1998]。这些参加决赛的算法在又一次更深入的评论期(第 2 轮)得到进一步的分析。在第 2 轮中,要征询对候选算法的各方面的评论和分析,这些方面包括密码分析、智能性、所有 AES 决赛候选算法的剖析、综合评价及有关实现问题,但并不限于上面所述的方面。2000 年 5 月 15 日,第 2 轮公众分析期结束以后,NIST 研究了所有可得到的信息以便为 AES 做出选择。2000 年 10 月 2 日,NIST 宣布它已经选中了 Rijndael 来建议作为 AES。

Rijndael 是由两个比利时密码学家 Daemen 和 Rijmen 共同设计的。

4.7.1 Rijndael 密码概述

Rijndael 是分组长度和密钥长度均可变的分组密码, 密钥长度和分组长度可以独立指定为 128b、192b 或 256b。为简化起见, 只讨论密钥长度为 128b, 分组长度为 128b 时的情形。所限定的描述无损于 Rijndael 密码工作原理的一般性。

在这种情况下, 128b 的消息(明文, 密文)分组被分成 16 个字节(一个字节是 8b, 所以有 $128 = 16 \times 8$), 记为

$$\text{InputBlock} = m_0, m_1, \dots, m_{15}$$

密钥分组如下:

$$\text{InputKey} = k_0, k_1, \dots, k_{15}$$

内部数据结构的表示是一个 4×4 矩阵:

$$\text{InputBlock} = \begin{pmatrix} m_0 & m_4 & m_8 & m_{12} \\ m_1 & m_5 & m_9 & m_{13} \\ m_2 & m_6 & m_{10} & m_{14} \\ m_3 & m_7 & m_{11} & m_{15} \end{pmatrix}$$

$$\text{InputKey} = \begin{pmatrix} k_0 & k_4 & k_8 & k_{12} \\ k_1 & k_5 & k_9 & k_{13} \\ k_2 & k_6 & k_{10} & k_{14} \\ k_3 & k_7 & k_{11} & k_{15} \end{pmatrix}$$

同 DES (以及最现代的对称密钥分组密码) 一样, Rijndael 算法也是由基本的变换单位——“轮”多次迭代而成, 在消息分组长度和密钥分组均为 128b 的最小情况, 轮数是 10, 当消息长度和密钥长度变大时, 轮数也应该相应增加。有关内容请参阅[NIST 2001a]。

Rijndael 中的轮变换记为

$\text{Round}(\text{State}, \text{RoundKey})$

这里 State 是轮消息矩阵, 既被看做输入, 也被看做输出; RoundKey 是轮密钥矩阵, 它是由输入密钥通过密钥表导出的。一轮的完成将导致 State 的元素改变值(也就是改变它的状态)。对于加密(对应解密), 输入到第 1 轮中的 State 就是明文(对应密文)消息矩阵 InputBlock, 而最后一轮中输出的 State 就是密文(对应明文)消息矩阵。

轮(除了最后一轮)变换由 4 个不同的变换组成, 这些变换是将要介绍的内部函数:

```
Round(State, RoundKey) {
    SubBytes(State);
    ShiftRows(State);
    MixColumns(State);
    AddRoundKey(State, RoundKey);
}
```

最后一轮有点不同，记为

$\text{FinalRound}(\text{State}, \text{RoundKey})$

它等于不使用 Mixcolumns 函数的 $\text{Round}(\text{State}, \text{RoundKey})$ ，这类似于 DES 中最后一轮的情形，就是在输出的半数据分组之间再做一次交换。

轮变换是可逆的，以便于解密，相应的逆轮变换分别记为

$\text{Round}^{-1}(\text{State}, \text{RoundKey})$

和

$\text{FinalRound}^{-1}(\text{State}, \text{RoundKey})$

下面可看到 4 个内部函数都是可逆的。

4.7.2 Rijndael 密码的内部函数

现在介绍 Rijndael 密码的 4 个内部函数，因为每个内部函数都是可逆的，为了实现 Rijndael 的解密，只需要在相反的方向使用它们各自的逆就可以了，因此仅在加密方向来描述这些函数。

Rijndael 密码的内部函数是在有限域上实现的， F_2 上的所有多项式模不可约多项式

$$f(x) = x^8 + x^4 + x^3 + x + 1$$

就得到了这个域。明确地说，Rijndael 密码所用的域是 $F_2[x]_{x^8+x^4+x^3+x+1}$ ，这个域中的元素就是 F_2 上次数小于 8 的多项式，运算是模 $f(x)$ 运算，把这个域称为“Rijndael 域”。由于同构关系，经常用 F_{2^8} 来表示这个域，这个域中有 2^8 (256) 个元素。

在 Rijndael 密码中，一个消息分组（一个状态）和一个密钥分组被分成字节。这些字节可以看成是域元素并由将要描述的几个 Rijndael 内部函数所使用。

1. 内部函数 SubBytes (State)

这个函数为 State 的每一字节（也就是 x ）提供了一个非线性代换，任一非 0 字节 $x \in (F_{2^8})^*$ 被下面的变换所代换：

$$y = Ax^{-1} + b \quad (4-32)$$

这里

$$A = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \quad \text{和} \quad b = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$$

如果 x 是 0 字节, 那么 $y=b$ 就是 SubBytes 变换的结果。

注意在式 (4-32) 中变换的非线性仅仅来自于逆 x^{-1} , 如果这个变换直接作用于 x , 那么在式 (4-32) 中的仿射方程将绝对是线性的。

因为 8×8 常数矩阵 A 是可逆的 (也就是说, 它的行在 F_{2^8} 中是线性无关的), 所以在式 (4-32) 中的变换是可逆的, 因此函数 SubBytes (State) 是可逆的。

2. 内部函数 ShiftRows (State)

这个函数在 State 的每行上运算, 对于 128b 分组长度的情形, 它的变换如下:

$$\begin{pmatrix} S_{0,0} & S_{0,1} & S_{0,2} & S_{0,3} \\ S_{1,0} & S_{1,1} & S_{1,2} & S_{1,3} \\ S_{2,0} & S_{2,1} & S_{2,2} & S_{2,3} \\ S_{3,0} & S_{3,1} & S_{3,2} & S_{3,3} \end{pmatrix} \rightarrow \begin{pmatrix} S_{0,0} & S_{0,1} & S_{0,2} & S_{0,3} \\ S_{1,1} & S_{1,2} & S_{1,3} & S_{1,0} \\ S_{2,2} & S_{2,3} & S_{2,0} & S_{2,1} \\ S_{3,3} & S_{3,0} & S_{3,1} & S_{3,2} \end{pmatrix}. \quad (4-33)$$

这个运算实际上是一个换位密码, 它只是重排了元素的位置而不改变元素本身: 对于在第 $i(i=0,1,2,3)$ 行的元素, 位置重排就是“循环向右移动” $4-i$ 个位置。

既然换位密码仅仅重排行元素的位置, 那么这个变换当然是可逆的。

3. 内部函数 MixColumns (State)

这个函数在 State 的每列上作用, 所以对于式 (4-33) 中右边矩阵的 4 列 State, MixColumns(State) 迭代 4 次。下面只描述对一列的作用, 一次迭代的输出仍是一列。

首先, 令

$$\begin{pmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \end{pmatrix}$$

是式 (4-33) 中右边矩阵中的一列。注意, 为了表述清楚, 已经省略了列数。

把这一列表示为 3 次多项式:

$$s(x) = s_3x^3 + s_2x^2 + s_1x + s_0$$

注意到因为 $s(x)$ 的系数是字节, 也就是 F_{2^8} 中的元素, 所以这个多项式是在 F_{2^8} 上的, 因此不是 Rijndael 中的元素。

列 $s(x)$ 上的运算定义为将这个多项式乘以一个固定的 3 次多项式 $c(x)$, 然后模 $x^4 + 1$:

$$c(x) \cdot s(x) \pmod{x^4 + 1} \quad (4-34)$$

这里固定的多项式 $c(x)$ 是

$$c(x) = c_3x^3 + c_2x^2 + c_1x + c_0 = '03'x^3 + '01'x^2 + '01'x + '02'$$

$c(x)$ 的系数也是 F_{2^8} 中的元素 (以十六进制表示字节或域元素)。

注意到式 (4-34) 中的乘法不是 Rijndael 域中的运算: $c(x)$ 和 $s(x)$ 甚至不是 Rijndael 域中的元素。而且因为 $x^4 + 1$ 在 F_2 上可约 ($x^4 + 1 = (x+1)^4$), 在式 (4-34) 中的乘法甚至不是任何域中的运算。进行乘法模一个 4 次多项式的仅有的理由就是为了使运算输出一个 3 次多项式, 也就是说, 为了获得一个从一列 (3 次多项式) 到另一列 (3 次多项式)

的变换，这个变换可以看做是使用已知密钥的一个多表代换（乘积）密码。

可使用长除法来验证下面在 F_2 上计算的方程（注意到在这个环中减法与加法等同）：

$$x^i \pmod{x^4 + 1} = x^{i \bmod 4}$$

因此，在式（4-34）的乘积中， $x^i (i = 0, 1, 2, 3)$ 的系数一定是满足 $j + k = i \bmod 4$ 的 $c_j s_k$ 的和（这里 $j, k = 0, 1, 2, 3$ ），例如，在乘积中 x^2 的系数是

$$c_2 s_0 + c_1 s_1 + c_0 s_2 + c_3 s_3$$

因为乘法和加法都在 F_2 中，所以很容易验证式（4-34）中的多项式乘法可由下面的线性代数式给出

$$\begin{pmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{pmatrix} = \begin{pmatrix} c_0 & c_3 & c_2 & c_1 \\ c_1 & c_0 & c_3 & c_2 \\ c_2 & c_1 & c_0 & c_3 \\ c_3 & c_2 & c_1 & c_0 \end{pmatrix} \begin{pmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \end{pmatrix} = \begin{pmatrix} '02' & '03' & '01' & '01' \\ '01' & '02' & '03' & '01' \\ '01' & '01' & '02' & '03' \\ '03' & '01' & '01' & '02' \end{pmatrix} \begin{pmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \end{pmatrix} \quad (4-35)$$

进一步注意到，因为在 F_2 上 $c(x)$ 与 $x^4 + 1$ 是互素的，所以在 $F_2[x]$ 中逆 $c(x)^{-1} \pmod{x^4 + 1}$ 是存在的。这等价于说矩阵式（4-35）中的变换是可逆的。

4. 内部函数 AddRoundKey (State, RoundKey)

这个函数仅仅是逐字节、逐比特地将 RoundKey 中的元素与 State 中的元素相加，这里的“加”是 F_2 中的加法（也就是逐比特异或），是平凡可逆的，逆就是自身相“加”。

RoundKey 比特已经被列表，也就是说，不同轮的密钥比特是不同的，它们由使用一个固定的（非秘密的）“密钥表”方案的密钥导出，有关“密钥”表的细节请参阅[NIST 2001a, 2001b]。

到此为止，已经完成了 Rijndael 内部函数的描述，因此也完成了加密运算的描述。

5. 解密运算

综上所述，4 个内部函数都是可逆的，因此解密仅仅是在相反的方向反演加密，也就是说，运行

```
AddRoundKey(State, RoundKey)-1;  
MixColumns(State)-1;  
ShiftRows(State)-1;  
SubBytes(State)-1;
```

应当注意，它与 Feistel 密码不同：Feistel 密码的加密和解密可以使用同样的电路（硬件）和代码（软件），而 Rijndael 密码的加密和解密必须分别使用不同的电路和代码。

结束对 Rijndael 密码描述之前，对 4 个内部函数的功能给出一个小结。

（1）SubBytes 目的是为了得到一个非线性的代换密码。对于分组密码抗差分分析来说，非线性是一个重要的性质。

（2）ShiftRows 和 MixColumns 目的是获得明文消息分组在不同位置上的字节的混合。有代表性的比如，由于在自然语言和商业数据中包含的高冗余导致的明文消息在消息空间有一个低熵分布（也就是说，典型的明文集中在整个消息空间中的一个较小的子空间中），而消息分组中不同位置上的字节的混合导致了消息在整个消息空间中更广的分

布。这本质上就是香农提出的混合特性。

(3) AddRoundKey 给出了消息分布所需的秘密随机性。

这些函数重复多次（在 128b 密钥和数据长度的情形下，至少要重复 10 次）以后，就构成了 Rijndael 密码。

4.7.3 AES 密码算法

在 Rijndael 算法中，分组长度和密钥长度均能分别被指定为 128b、192b 或 256b。在高级加密标准规范中，密钥的长度可以使用三者中的任意一种，但分组长度只能是 128b。高级加密标准中众多参数与密钥长度（参见表 4-3）有关。在本章中，假定密钥的长度为 128b，这可能是使用最广泛的实现方式。图 4-13 中是 AES 的完整结构。

表 4-3 AES 的参数

密钥长度（word/byte/bit）	4/16/128	6/24/192	8/32/256
分组长度（word/byte/bit）	4/16/128	4/16/128	4/16/128
轮数	10	12	14
每轮的密钥长度（word/byte/bit）	4/16/128	4/16/128	4/16/128
扩展密钥长度（word/byte）	44/176	52/208	60/240

1. AES 加密算法

加密算法的输入分组和解密算法的输出分组均为 128b。在 FIPS PUB 197 中，输入分组是用以字节为单位的正方形矩阵描述的。且该分组被复制到 State 数组，这个数组在加密或解密的每个阶段都会被改变。在执行了最后的阶段后，State 被复制到输出矩阵中。这些操作在图 4-13（a）中描述。同样，128b 的密钥也是用以字节为单位的矩阵描述的。然后这个密钥被扩展成一个以字为单位的密钥序列数组；每个字由 4 个字节组成，128b 的密钥最终扩展为 44 个字的序列（参见图 4-13（b））。注意在矩阵中字节排列顺序是从



(a) 输入、State数据组和输出



(b) 密钥和扩展密钥

图 4-13 AES 的数据结构

上到下、从左到右排列的。加密算法中每个 128b 分组输入的前 4 个字节被按顺序放在了 in 矩阵的第 1 列，接着的 4 个字节放在了第 2 列，等等。同样，扩展密钥的前 4 个字（1 个字）被放在 w 矩阵的第 1 列。

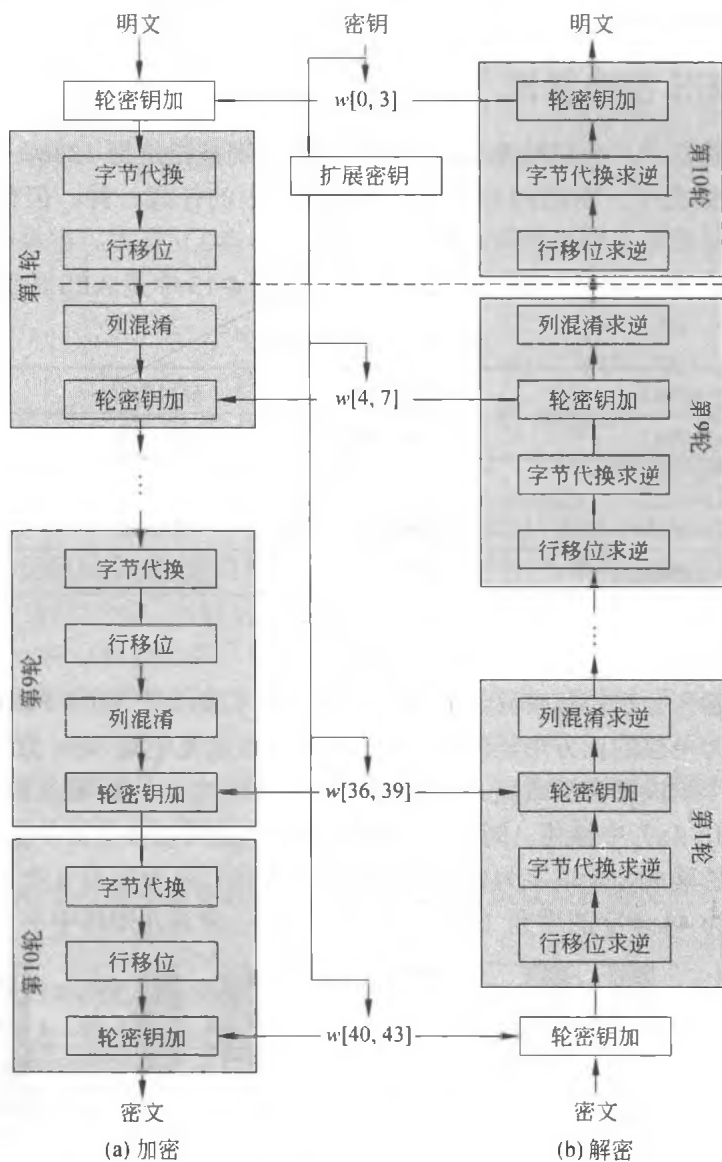


图 4-14 AES 的加密与解密

2. AES 解密算法

AES 的解密算法和加密算法不同 (参见图 4-14)。尽管在加密和解密中密钥扩展的形式一样，但在解密中变换的顺序与加密中变换的顺序不同。其缺点在于对同时需要加密和解密的应用而言，需要两个不同的软件或固件模块。然而，解密算法的一个等价版本与加密算法有同样的结构。这个版本与加密算法的变换顺序相同 (用逆变换取代正向变换)。为了达到这个目标，需要对密钥扩展进行改进。

两处改进使解密算法的结构与加密算法的结构一致。在加密过程中,其轮结构为字节代换、行移位、列混淆和轮密钥相加。在标准的解密过程中,其轮结构为逆向行移位、逆向字节代换、轮密钥加和逆向列混淆。因此,在解密轮中的前两个阶段应交换,后两个阶段也需要交换。

4.7.4 AES 的密钥扩展

1. 密钥扩展算法

AES 密钥扩展算法的输入值是 4 个字 (16B), 输出值是一个 44 个字 (176B) 的一维线性数组。这足以以为算法中的初始 Add Round Key 阶段和其他 10 轮中的每一轮提供 4 个字 (word) 的轮密钥。下面用伪代码描述了这个扩展:

```

KeyExpansion (byte key[16], word w[44])
{
    word temp
    for (i=0; i<4; i++)          w[i]=      (key[4*i],      key[4*i+1],
                                     key[4*i+2],
                                     key[4*i+3]);

    for (i=4; i<44; i++)
    {
        temp=w[i-1];
        if (i mod 4=0)            temp=SubWord (RotWord (temp))
                                ⊗ Rcon[i/4];

        w[i]=w[i-4] ⊗ temp
    }
}

```

输入密钥直接被复制到扩展密钥数组的前 4 个字。然后每次用 4 个字填充扩展密钥数组余下的部分。在扩展密钥数组中, $w[i]$ 的值依赖于 $w[i-1]$ 和 $w[i-4]$ 。在 4 个情形中, 3 个使用了异或。对 w 数组中下标为 4 的倍数元素采用了更复杂的函数来计算。图 4-15 阐明了如何计算扩展密钥数组的前 8 个字节, 其中使用符号 g 来表示这个复杂函数:

(1) 字循环的功能是使一个字中的 4 个字节循环左移一个字节。即将输入字 $[b_0, b_1, b_2, b_3]$ 变换成 $[b_1, b_2, b_3, b_0]$ 。

(2) 字节代换利用 S 盒对输入字中的每个字节进行字节代换。

(3) 步骤 1 和步骤 2 的结果再与轮常量 $Rcon[j]$ 相异或。

轮常量是一个字, 这个字最右边的 3 个字节总为 0。因此与 $Rcon$ 中的一个字相异或,

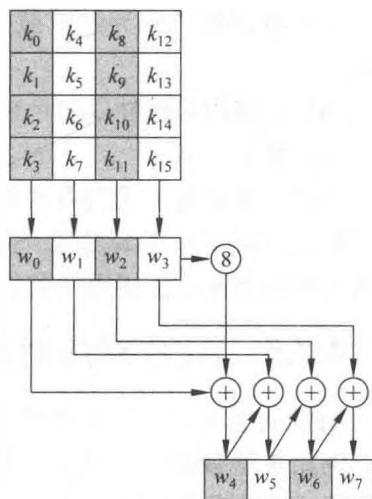


图 4-15 AES 密钥扩展

其结果只是与该字最左边的那个字节相异或。每轮的轮常量均不同，其定义为 $Rcon[j]=(RC[j], 0, 0, 0)$ ，其中 $RC[1]=1$ ， $RC[j]=2 \cdot RC[j-1]$ （乘法是定义在域 $GF(2^8)$ ）。 $RC[j]$ 的值按十六进制表示为

j	1	2	3	4	5	6	7	8	9	10
$RC[j]$	01	02	04	08	10	20	40	80	1B	36

例如，假设第 8 轮的轮密钥为

EA D2 73 21 B5 8D BA D2 31 2B F5 60 7F 8D 29 2F

那么第 9 轮的轮密钥的前 4 个字节（第 1 列）能按如下的方式计算：

i (十进制)	temp	RotWord 后	SubWord 后	Rcon(9)	与 Rcon 异或后	$w[i-4]$	$w[i]=temp \oplus w[i-4]$
36	7F8D292F	8D292F7F	5DA515D2	1B000000	46A515D2	EAD27321	AC7766F3

2. 评价

Rijndael 的开发者设计了密钥扩展算法来防止已有的密码分析攻击。使用与轮相关的轮常量是为了防止不同轮中产生的轮密钥的对称性或相似性。文献[Deamen 等 1999]中使用的标准如下：

- (1) 知道密钥或轮密钥的部分位不能计算出轮密钥的其他位。
- (2) 它是一个可逆的变换（即知道扩展密钥中任何连续的 N_k 个字能够重新产生整个扩展密钥（ N_k 是构成密钥所需的字数））。
- (3) 能够在各种处理器上有效地执行。
- (4) 使用轮常量来排除对称性。
- (5) 将密钥的差异性扩散到轮密钥中的能力；即密钥的每位能影响到轮密钥的一些位。
- (6) 足够的非线性以防止轮密钥的差异完全由密钥的差异所决定。
- (7) 易于描述。

作者并未量化上述列表的第一点，但指出了如果你知道的密钥或在某个轮密钥中少于 N_k 个连续字，那么将难于构造出其余的未知位。知道密钥的位数量越少就越难于重构出或推测出密钥扩展中的其他位。

4.7.5 AES 对应用密码学的积极影响

AES 的引入又为应用密码学带来几个积极的变化。首先，随着 AES 的出现，多重加密，例如 3 重 DES，已成为不必要的了。加长和可变的密钥及 128b，192b 和 256b 的数据分组长度为各种应用要求提供了大范围可选的安全强度。由于多重加密多次使用密钥，那么避免使用多重加密就意味着实用中必须使用的密钥数目的减少，因此可以简化安全协议和系统的设计。

其次, AES 的广泛使用将导致同样强度的新的杂凑函数的出现。在某些情形下, 分组加密算法与杂凑函数密切相关(见第6章), 分组加密算法经常被用来作为单向杂凑函数, 这已经成为一种标准应用。UNIX 操作系统的登录认证协议就是一个著名的例子。另外, 利用分组加密算法可以实现单向杂凑函数。实用中, 杂凑函数也经常被用来为分组密码算法生成密钥的伪随机数函数。由于 AES 可变、加长的密钥和数据分组长度, 将需要相同输出长度的杂凑函数。然而, 由于平方根攻击(生日攻击), 杂凑函数的长度应该是分组密码密钥或数据分组长度的两倍, 因此将需要与 128b, 192b 和 256b 的 AES 长度相匹配的 256b, 384b 和 512b 输出长度的新的杂凑函数。ISO/IEC 现在正在进行杂凑函数 SHA-256, SHA-384 和 SHA-512 的标准化工作[ISO/IEC 2001]。

正如 DES 标准吸引了许多试图攻破该算法的密码分析家的注意, 并促进了分组密码分析的认识水平的发展一样, 作为新的分组密码标准的 AES 也将再次引起分组密码分析中的高水平研究, 这必将使得人们对该领域的认识水平得到进一步的提高。

4.8

中国商用分组密码算法 SM4

2006 年我国国家密码管理局公布了无线局域网产品使用的 SM4 (原名 SMS4) 密码算法, 这是我国第一次公布自己的商用密码算法。这一举措标志着我国商用密码管理更加科学化、规范化和国际化, SM4 的公布在我国商用密码的产业发展中具有里程碑意义。

4.8.1 SM4 密码算法

SM4 是分组长度和密钥长度均为 128 比特的 32 轮迭代分组密码算法, 它以字节和字为单位对数据进行处理。SM4 解密算法与加密算法的结构相同, 只是轮密钥的使用顺序相反, 解密轮密钥是加密轮密钥的逆序。

1. 基本运算

(1) SM4 使用模 2 加和循环移位运算

① 模 2 加: $\oplus$, 32 比特异或运算;

② 循环移位: $\lll i$, 32 比特循环左移 i 位。

(2) 置换运算: S 盒

S 盒是一种固定的 8 比特输入、8 比特输出的置换运算, 记为 $Sbox(\cdot)$, 它的密码学作用是起混淆作用。 S 盒的置换运算如表 4-4 所示。例如, S 盒的输入为 $9a$, 则 S 盒的输出为表 4-4 中第 9 行与第 a 列的交点处的值 32。即, $Sbox(9a)=32$ 。

(3) 非线性变换 τ

τ 是一种以字为单位的非线性变换, 它由 4 个并行的 S 盒构成。设输入为 $A = (a_0, a_1, a_2, a_3)$, 输出为 $B = (b_0, b_1, b_2, b_3)$, 则

$$B = \tau(A) = (Sbox(a_0), Sbox(a_1), Sbox(a_2), Sbox(a_3))$$

(4) 线性变换 L

L 是以字为单位的线性变换，它的输入、输出都是 32 位的字。其密码学的作用是起扩散作用。设输入为字 B ，输出为字 C ，则

$$C = L(B) = B \oplus (B \ll 2) \oplus (B \ll 10) \oplus (B \ll 18) \oplus (B \ll 24)$$

(5) 合成变换 T

T 由非线性变换 τ 和线性变换 L 复合而成，数据处理单位是字，即 $T(\cdot) = L(\tau(\cdot))$ 。它在密码学中起到了混淆和扩散的作用，因而可以提高安全性。

表 4-4 盒表

		低 位															
		0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
高 位	0	d6	90	e9	fe	cc	e1	3d	b7	16	b6	14	c2	28	fb	2c	05
	1	2b	67	9a	76	2a	be	04	c3	aa	44	13	26	49	86	06	99
	2	9c	42	50	f4	91	ef	98	7a	33	54	0b	43	ed	cf	ac	62
	3	e4	b3	1c	a9	c9	08	e8	95	80	df	94	fa	75	8f	3f	a6
	4	47	07	a7	fc	f3	73	17	ba	83	59	3c	19	e6	85	4f	a8
	5	68	6b	81	b2	71	64	da	8b	f8	eb	0f	4b	70	56	9d	35
	6	1e	24	0e	5e	63	58	d1	a2	25	22	7c	3b	01	21	78	87
	7	d4	00	46	57	9f	d3	27	52	4c	36	02	e7	a0	c4	c8	9e
	8	ea	bf	8a	d2	40	c7	38	b5	a3	f7	f2	ce	f9	61	15	a1
	9	e0	ae	5d	a4	9b	34	1a	55	ad	93	32	30	f5	8c	b1	e3
	a	1d	f6	e2	2e	82	66	ca	60	c0	29	23	ab	0d	53	4e	6f
	b	d5	db	37	45	de	fd	8e	2f	03	ff	6a	72	6d	6c	5b	51
	c	8d	1b	af	92	bb	dd	bc	7f	11	d9	5c	41	1f	10	5a	d8
	d	0a	c1	31	88	a5	cd	7b	bd	2d	74	d0	12	b8	e5	b4	b0
	e	89	69	97	4a	0c	96	77	7e	65	b9	f1	09	c5	6e	c6	84
	f	18	f0	7d	ec	3a	dc	4d	20	79	ee	5f	3e	d7	cb	39	48

(6) 轮函数 F

轮函数 F 采用非线性迭代结构，以字为单位进行加密运算，称一次迭代运算为一轮变换。设 F 的输入为 (X_0, X_1, X_2, X_3) ，4 个 32 位字；轮密钥为 rk ， rk 也是一个 32 位字。轮函数的运算式为：

$$F(X_0, X_1, X_2, X_3, rk) = X_0 \oplus T(X_1 \oplus X_2 \oplus X_3 \oplus rk)$$

简记 $B = (X_1 \oplus X_2 \oplus X_3 \oplus rk)$ ，再由合成变换 T 可展开为非线性变换 τ 与线性变换 L ，可以得到

$$F(X_0, X_1, X_2, X_3, rk) = X_0 \oplus [Sbox(B)] \oplus [Sbox(B) \ll 2] \oplus [Sbox(B) \ll 10] \oplus [Sbox(B) \ll 18] \oplus [Sbox(B) \ll 24]$$

2. 加密算法

SM4 加密算法的数据分组长度为 128 比特，密钥长度也为 128 比特。加密算法采用

32 轮迭代结构, 每一轮迭代使用一个轮密钥。完整的加密过程包括加密算法和反序变换两部分, 如图 4-16 所示。

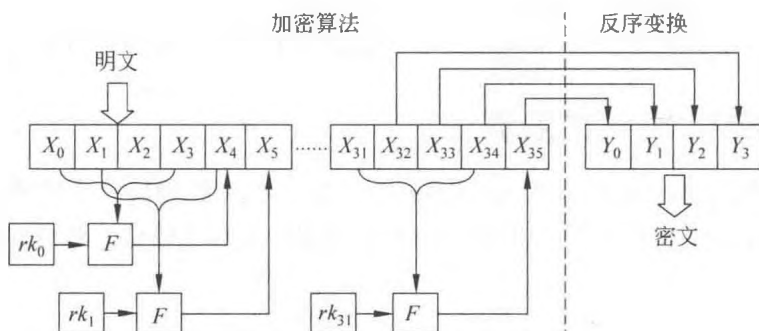


图 4-16 SM4 的加密算法与反序变换

(1) 加密算法

设输入明文为 (X_0, X_1, X_2, X_3) , 4 个 32 位字。输入轮密钥为 $rk_i, i = 0, 1, \dots, 31$, 共 32 个字。加密算法可描述如下:

$$X_{i+4} = F(X_i, X_{i+1}, X_{i+2}, X_{i+3}, rk_i) = X_i \oplus T(X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i)$$

结合图 4-16, SM4 每一轮加密处理 4 个字 $(X_i, X_{i+1}, X_{i+2}, X_{i+3})$, 并产生一个字的中间密文 X_{i+4} , 这个中间密文与前 3 个字 $(X_{i+1}, X_{i+2}, X_{i+3})$ 拼接在一起供下一轮加密处理。这样的加密处理共迭代 32 轮, 最终产生出 4 个字的准密文 $(X_{32}, X_{33}, X_{34}, X_{35})$ 。

(2) 反序变换 R

反序变换 R 的输入是准密文 $(X_{32}, X_{33}, X_{34}, X_{35})$, 输出是密文 (Y_0, Y_1, Y_2, Y_3) , 具体变换如下:

$$R(X_{32}, X_{33}, X_{34}, X_{35}) = (X_{35}, X_{34}, X_{33}, X_{32}) = (Y_0, Y_1, Y_2, Y_3)$$

3. 解密算法

SM4 的解密与加密的流程相同, 包括解密算法和反序变换两部分, 不同的仅是轮密钥的使用顺序相反, 加密时轮密钥使用顺序为 $(rk_0, rk_1, \dots, rk_{31})$, 则解密时轮密钥的使用顺序为 $(rk_{31}, rk_{30}, \dots, rk_0)$, 如图 4-17 所示。

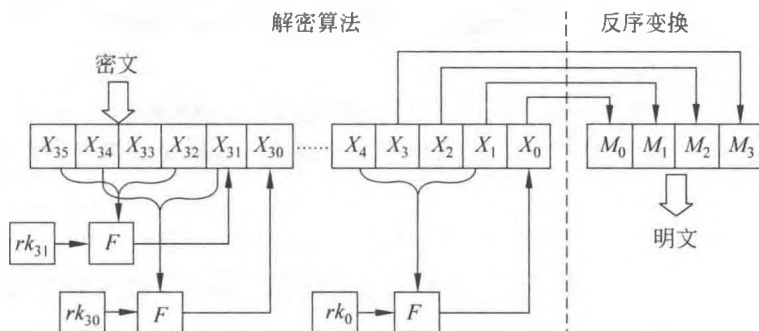


图 4-17 SM4 的解密算法与反序变换

(1) 解密算法

为了便于读者与加密算法对照, 解密算法中仍然使用 X_i 表示密文, $i = 31, 30, \dots, 1, 0$ 。

$$X_i = F(X_{i+4}, X_{i+3}, X_{i+2}, X_{i+1}, rk_i) = X_{i+4} \oplus T(X_{i+3} \oplus X_{i+2} \oplus X_{i+1} \oplus rk_i)$$

(2) 反序变换 R

设输出的明文为 (M_0, M_1, M_2, M_3) ，反序变换如下：

$$R(X_3, X_2, X_1, X_0) = (X_0, X_1, X_2, X_3) = (M_0, M_1, M_2, M_3)$$

4.8.2 SM4 密钥扩展算法

SM4 的加算法中，采用了 32 轮迭代运算，每一轮迭代使用一个轮密钥，因此总共需要 32 个轮密钥，这些轮密钥由加密密钥通过密钥扩展算法生成。密钥扩展中使用了以下两组参数：

(1) 系统参数 FK

系统参数 FK 的取值，采用十六进制表示：

$$FK_0 = (A3B1BAC6), FK_1 = (56AA3350), FK_2 = (677D9197), FK_3 = (B27022DC)$$

(2) 固定参数 CK

CK_i 是一个字，密钥扩展中共使用了 32 个 CK_i 。设 $ck_{i,j}$ 为 CK_i 的第 j 个字节 ($i = 0, 1, \dots, 31; j = 0, 1, 2, 3$)，即 $CK_i = (ck_{i,0}, ck_{i,1}, ck_{i,2}, ck_{i,3})$ ，则

$$ck_{i,j} = (4i + j) \times 7 \pmod{256}$$

这 32 个固定参数 CK_i 的十六进制表示如下：

00070e15	1c232a31,	383f464d,	545b6269,
70777e85,	8c939aa1,	a8afb6bd,	c4cbd2d9,
e0e7eef5,	fc030a11,	181f262d,	343b4249,
50575e65,	6c737a81,	888f969d,	a4abb2b9,
c0c7ced5,	dce3eaf1,	f8ff060d,	141b2229,
30373e45,	4c535a61,	686f767d,	848b9299,
a0a7aeb5,	bcc3cad1,	d8dfe6ed,	f4fb0209,
10171e25,	2c333a41,	484f565d,	646b7279.

设密钥扩展算法中输入的加密密钥为 $MK = (MK_0, MK_1, MK_2, MK_3)$ ，输出轮密钥为 rk_i ， $i = 0, 1, \dots, 30, 31$ ，中间数据为 K_i ， $i = 0, 1, \dots, 34, 35$ 。密钥扩展算法分为以下两步：

(1) $(K_0, K_1, K_2, K_3) = (MK_0 \oplus FK_0, MK_1 \oplus FK_1, MK_2 \oplus FK_2, MK_3 \oplus FK_3)$

(2) 对于 $i = 0, 1, \dots, 30, 31$ 执行以下操作：

$$rk_i = K_{i+4} = K_i \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i)$$

注意这里的 T' 变换与加密算法轮函数的 T 基本相同，只是将其中的线性变换 L 修改为 L' ：

$$L'(B) = B \oplus (B \lll 13) \oplus (B \lll 23)$$

SM4 密钥扩展算法也需要采用 32 轮的迭代处理。算法中涉及的非线性变换将极大地提高密钥扩展的安全性。

4.8.3 SM4 的安全性

SM4 密码算法是我国官方公布的第一个商用密码算法 (<http://www.oscca.gov.cn>), 其主要目的是加密与保护静态储存和传输信道中的数据, 它广泛应用于无线局域网产品。

从算法设计上看, SM4 在计算过程中增加了非线性变换, 理论上能大大加强算法的安全性, S 盒的引入使得该算法在非线性度、运算速度、差分均匀性、自相关性等主要密码学指标方面都具有相当的优势。近年来, 国内外密码学者对 SM4 进行了充分的分析与实验, 例如, 利用复合域实现 S 盒以降低硬件开销; 对 S 盒进行差分故障攻击, 以显示 SM4 抵抗故障攻击的能力; 对国密 SM4 与 SM2 混合密码算法进行研究与实现, 以提高加密速度与降低密钥管理成本。这些研究致力于 SM4 的低复杂度实现、混合加密技术的商用化、SM4 抗攻击能力的增强等方面, 这些研究成果对我们改进 SM4 密码和设计新密码都是有帮助的。至今, 我国国家密码管理局仍然支持 SM4 密码, 它的广泛应用为确保我国信息安全做出了积极贡献。

4.9 分组密码的工作模式

分组密码将消息作为数据分组处理(加密或解密)。一般来讲, 大多数消息(也就是一个消息串)的长度大于分组密码的消息分组长度, 长的消息串被分成一系列的连续排列的消息分组, 密码机一次处理一个分组。

人们在设计了基本的分组密码算法之后, 紧接着设计了许多不同的运行模式。这些运行模式(除去其中平凡的情形)为密文分组提供了几个人们希望得到的性质, 例如, 增加分组密码算法的不确定性(随机性); 将明文消息添加到任意长度(使得密文长度不必与相应的明文长度相关); 错误传播的控制; 流密码的密钥流生成等。

这里描述 5 个常用的运行模式, 它们是电码本(ECB)模式、密码分组链接(CBC)模式、输出反馈(OFB)模式、密码反馈(CFB)模式和计数器(CTR)模式。这些描述是根据最近发布的 NIST 的建议书[NIST 2001b] 做出的。

在描述中, 将使用下面的记号。

- (1) $E()$: 基本分组密码的加密算法。
- (2) $D()$: 基本分组密码的解密算法。
- (3) n : 基本分组密码算法的消息分组的二进制长度(在所有考虑的分组密码中, 明文和密文消息空间是一样的, 所以 n 既是分组密码算法输入的分组长度, 也是输出的分组长度)。

(4) $P_1, P_2, \dots, P_m$: 输入到运行模式中明文消息的 m 个连续分段。

① 第 m 分段的长度可能小于其他分段的长度, 在这种情况下, 可对第 m 分段添加, 使其与其他分段长度相同。

② 在某些运算模式中, 消息分段的长度等于 n (分组长度), 而在其他运算模式中,

消息分段的长度是任意小于或等于 n 的正数。

(5) $C_1, C_2, \dots, C_m$ ：从运算模式输出的密文消息的 m 个连续分段。

(6) $\text{LSB}_u(B), \text{MSB}_v(B)$ ：分别是分组 B 中最低 u 位比特和最高 v 位比特，例如：

$$\text{LSB}_2(1010011) = 11, \text{MSB}_5(1010011) = 10100$$

(7) $A \| B$ ：数据分组 A 和 B 的链接，例如：

$$\text{LSB}_2(1010011) \| \text{MSB}_5(1010011) = 11 \| 10100 = 1110100$$

4.9.1 电码本模式

对一系列连续排列的消息段进行加密（或解密）的一个最直接方式就是对它们逐个加密（或解密）。在这种情况下，消息分段恰好是消息分组。由于类似于在电报密码本中指定码字，故给这个自然而简单的方法起了一个正式的名字：电码本模式（ECB），如图 4-18 所示。ECB 模式定义如下：

ECB 加密 $C_i \leftarrow E(P_i), i = 1, 2, \dots, m$ 。

ECB 解密 $P_i \leftarrow D(C_i), i = 1, 2, \dots, m$ 。

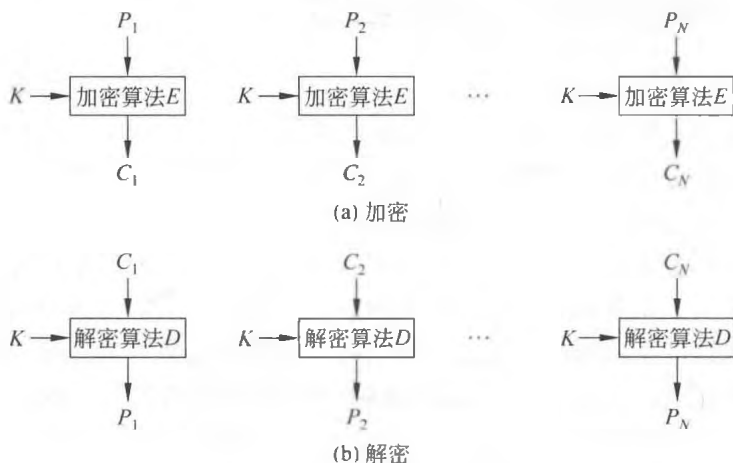


图 4-18 电码本（ECB）模式

ECB 模式是确定性的，也就是说，如果在相同的密钥下将 $P_1, P_2, \dots, P_m$ 加密两次，那么输出的密文分组也是相同的。在应用中，数据通常有部分可猜测的信息，例如，薪水的数目就有一个可猜测的范围。如果明文消息是可猜测的，那么由确定性加密方案得到的密文就会使攻击者通过使用试凑法猜测出明文，例如，如果知道由 ECB 模式加密产生的密文是一个薪水数字，那么攻击者只需少量的试验就可以恢复出这个数字。通常不希望使用确定性密码，因此在大多数应用中不要使用 ECB 模式。

4.9.2 密码分组链接模式

密码分组链接（CBC）运行模式是用于一般数据加密的一个普通的分组密码算法。使用 CBC 模式，输出是 n 个密码分组的一个序列，这些密码分组链接在一起使得每个密码分组不仅依赖于所对应的原文分组，而且依赖于所有以前的数据分组。CBC 模式进行

如下运算:

CBC 加密 输入: $IV, P_1, P_2, \dots, P_m$; 输出: $IV, C_1, C_2, \dots, C_m$;

$$C_0 \leftarrow IV;$$

$$C_i \leftarrow E(P_i \oplus C_{i-1}), i = 1, 2, \dots, m;$$

CBC 解密 输入: $IV, C_1, C_2, \dots, C_m$; 输出: $P_1, P_2, \dots, P_m$;

$$C_0 \leftarrow IV;$$

$$P_i \leftarrow D(C_i) \oplus C_{i-1}, i = 1, 2, \dots, m。$$

第一个密文分组 C_1 的计算需要一个特殊的输入分组 C_0 , 习惯上称之为“初始向量”(IV)。IV 是一个随机的 nb 分组, 每次会话加密时都要使用一个新的随机 IV, 由于 IV 可看成密文分组, 因此无须保密, 但一定是不可预知的。由加密过程知道, 由于 IV 的随机性, 第一个密文分组 C_1 被随机化, 同样, 依次后续的输出密文分组都将被前面紧接着的密文分组随机化, 因此, CBC 模式输出的是随机化的密文分组。发送给接收者的密文消息应该包括 IV。因此, 对于 m 个分组的明文, CBC 模式将输出 $m+1$ 个密文分组。

令 $Q_1, Q_2, \dots, Q_m$ 是对密文分组 $C_0, C_1, C_2, \dots, C_m$ 解密得到的数据分组输出, 则由

$$Q_i = D(C_i) \oplus C_{i-1} = (P_i \oplus C_{i-1}) \oplus C_{i-1} = P_i$$

可知, 它确实正确地进行了解密。图 4-19 给出了 CBC 模式的图示。



图 4-19 密码分组链接模式

4.9.3 密码反馈模式

密码反馈 (CFB) 运行模式的特点在于反馈相继的密码分段, 这些分段从模式的输出返回作为基础分组密码算法的输入。消息 (明文或密文) 分组长为 s , 其中 $1 \leq s \leq n$ 。CFB 模式要求 IV 作为初始的 nb 随机输入分组, 因为在系统中 IV 是在密文的位置中, 所以它不必保密。

CFB 模式有如下的运算:

CFB 加密 输入: $IV, P_1, P_2, \dots, P_m$; 输出: $IV, C_1, C_2, \dots, C_m$;

$$I_1 \leftarrow IV;$$

$$I_i \leftarrow \text{LSB}_{n-s}(I_{i-1}) \parallel C_{i-1} \quad i = 2, 3, \dots, m;$$

$$O_i \leftarrow E(I_i) \quad i = 1, 2, \dots, m;$$

$$C_i \leftarrow P_i \oplus \text{MSB}_s(O_i) \quad i = 1, 2, \dots, m。$$

CFB 解密 输入: $IV, C_1, C_2, \dots, C_m$; 输出: $P_1, P_2, \dots, P_m$;

$$I_1 \leftarrow IV;$$

$$I_i \leftarrow \text{LSB}_{n-s}(I_{i-1}) \parallel C_{i-1} \quad i = 2, 3, \dots, m;$$

$$O_i \leftarrow E(I_i) \quad i = 1, 2, \dots, m;$$

$$P_i \leftarrow C_i \oplus \text{MSB}_s(O_i) \quad i = 1, 2, \dots, m。$$

在 CFB 模式中，基本分组密码的加密函数用在加密和解密的两端。因此，基本密码函数 E 可以是任意（加密的）单向变换，例如单向杂凑函数。CFB 模式可以考虑作为流密码的密钥流生成器，加密变换是作用在密钥流和消息分段之间的弗纳姆密码。与 CBC 模式类似，密文分段是前面所有的明文分段的函数值和 IV。图 4-20 为 CFB 模式的图示。

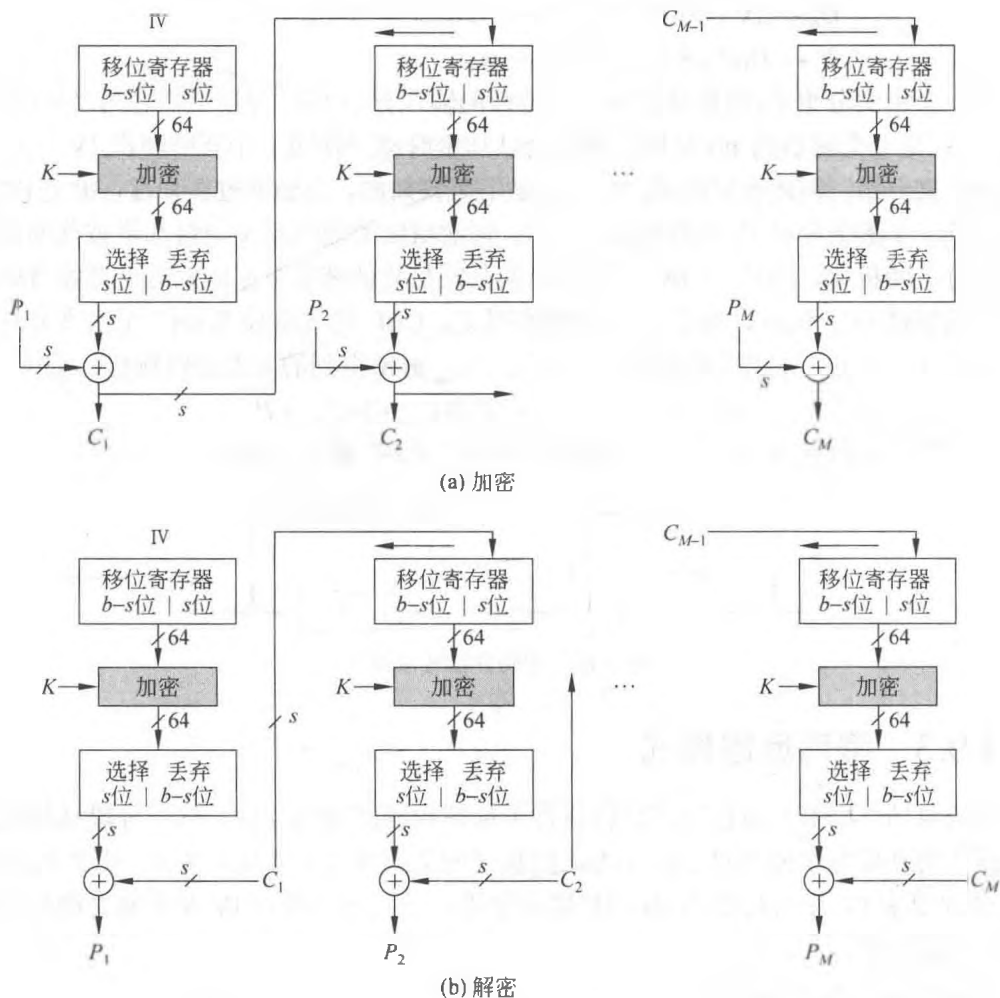


图 4-20 密码反馈模式

4.9.4 输出反馈模式

输出反馈（OFB）运行模式的特点是将基本分组密码的连续输出分组回送回去。这些反馈分组构成了一个比特串，被用做弗纳姆密码的密钥流的比特串，就是密钥流与明文分组相异或。OFB 模式要求 IV 作为初始的随机 nb 输入分组。因为在系统中，IV 是在密文的位置中，所以它不需要保密。OFB 模式运算如下。

OFB 加密 输入：IV, $P_1, P_2, \dots, P_m$ ；输出：IV, $C_1, C_2, \dots, C_m$ ；

$$I_1 \leftarrow \text{IV}；$$

$$I_i \leftarrow O_{i-1} \quad i = 2, 3, \dots, m;$$

$$O_i \leftarrow E(I_i) \quad i = 1, 2, \dots, m;$$

$$C_i \leftarrow P_i \oplus O_i \quad i = 1, 2, \dots, m。$$

OFB 解密 输入: $IV, C_1, C_2, \dots, C_m$; 输出: $P_1, P_2, \dots, P_m$;

$$I_1 \leftarrow IV;$$

$$I_i \leftarrow O_{i-1} \quad i = 2, 3, \dots, m;$$

$$O_i \leftarrow E(I_i) \quad i = 1, 2, \dots, m;$$

$$P_i \leftarrow C_i \oplus O_i \quad i = 1, 2, \dots, m。$$

在 OFB 模式中, 加密和解密是相同的: 将输入消息分组与由反馈电路生成的密钥流相异或。反馈电路实际上构成了一个有限状态机, 其状态完全由基础分组密码算法的加密密钥和 IV 决定。所以, 如果密码分组发生了传输错误, 那么只有相应位置上的明文分组会发生错乱, 因此, OFB 模式适宜不可能重发的消息加密, 如无线电信号。与 CFB 模式类似, 基础分组密码算法可用加密的单向杂凑函数代替。图 4-21 为 OFB 模式的图示。

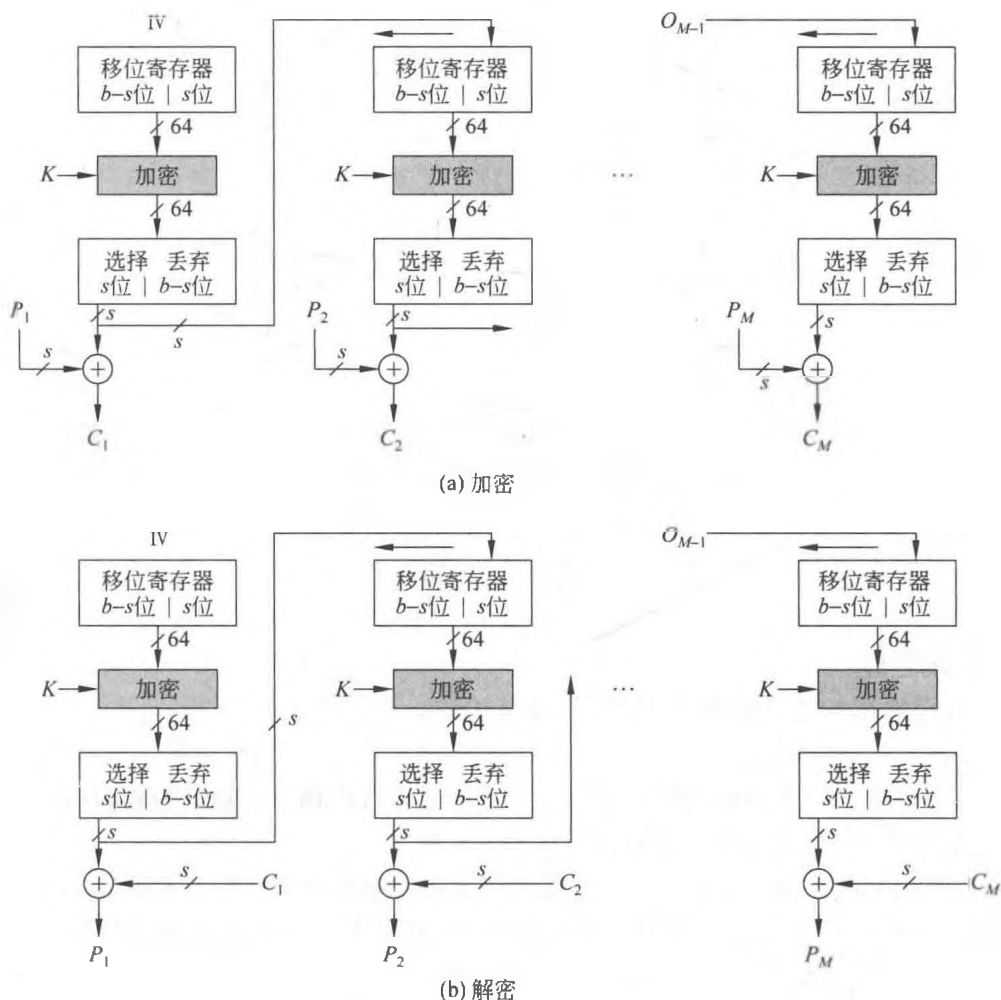


图 4-21 输出反馈模式 (加密和解密)

4.9.5 计数器模式

计数器（CTR）模式的特征是，将计数器从初始值开始计数所得到的值馈送给基础分组密码算法。随着计数的增加，基础分组密码算法输出连续的分组来构成一个比特串，该比特串被用做弗纳姆密码的密钥流，也就是密钥流与明文分组相异或。CTR 模式运算如下（这里 Ctr_1 是计数器初始的非保密值）。

CTR 加密 输入： $\text{Ctr}_1, P_1, P_2, \dots, P_m$ ；输出： $\text{Ctr}_1, C_1, C_2, \dots, C_m$ ；

$$C_i \leftarrow P_i \oplus E(\text{Ctr}_i), i = 1, 2, \dots, m。$$

CTR 解密 输入： $\text{Ctr}_1, C_1, C_2, \dots, C_m$ ；输出： $P_1, P_2, \dots, P_m$ ；

$$P_i \leftarrow C_i \oplus E(\text{Ctr}_i), i = 1, 2, \dots, m。$$

因为没有反馈，CTR 模式的加密和解密能够同时进行，这是 CTR 模式比 CFB 模式和 OFB 模式优越的地方。图 4-22 为 CTR 模式的图示。

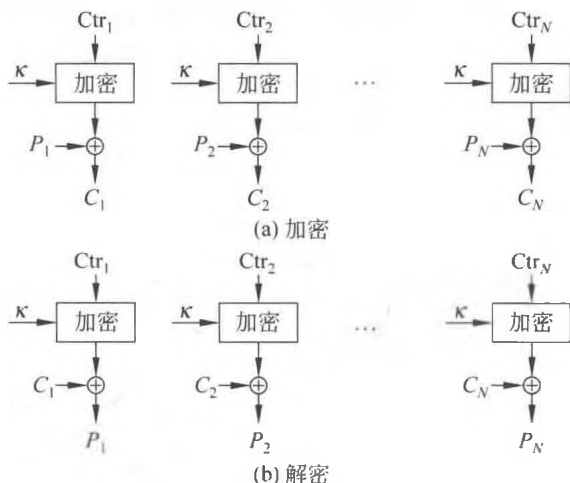


图 4-22 计数器（CTR）模式

习 题

一、填空题

1. 密码体制的语法定义由以下 6 部分构成：_____、_____、_____、_____、_____和_____。
2. 单（私）钥加密体制的特点是_____，所以人们通常也称其为对称加密体制。
3. 古典密码有两个基本工作原理：_____和_____。
4. 对明文消息的加密有两种：一种是将明文消息按照字符（如二元数字）逐位地加密，称为_____；另一种是将明文消息分组（含有多个字符），逐组地进行加密，称为_____。
5. 在理论上，加密信息的安全性不取决于_____的保密，而取决于_____的保密。

6. 美国数据加密标准 DES 的密钥长度为_____位, 分组长度为_____位。
7. 新一代数据加密标准 AES 的密钥长度是_____位, 分组长度是_____位。
8. A5 是欧洲蜂窝移动电话系统中采用的加密算法, 用于_____到_____线路上的加密。A5 的唯一缺点是_____。
9. 试列举 5 种常用的分组密码算法: _____、_____、_____、_____和_____。
10. 分组密码常用的工作模式有_____、_____、_____、_____和_____。
11. 祖冲之密码本质上是一个密钥序列产生算法, 其输入为_____比特的初始密钥和_____比特的初始向量, 输出为_____比特的密钥字序列。其逻辑上分为 3 层, 分别是_____、_____和_____。
12. SM4 密码的分组长度和密钥长度分别为_____和_____。加密算法采用_____轮迭代处理。

二、思考题

1. 加密算法为什么不应该包含秘密设计部分? 从理论上讲, 数据的保密是取决于算法的保密还是密钥的保密? 为什么?
2. 弗纳姆密码是一种代换密码吗? 它是单表代换还是多表代换?
3. 弗纳姆密码和一次一密体制的不同之处是什么?
4. 为什么说一次一密加密抗窃听是无条件安全的?
5. 虽然简单代换密码和换位密码对频度分析攻击是十分脆弱的, 为什么它们仍被广泛使用在现代加密方案和密码协议中?
6. 流密码是单钥体制还是双钥体制? 它与分组密码的区别是什么?
7. 现代密码通常是由几个古典密码技术结合起来构造的。在 DES 和 AES 中找出采用了下述 3 种密码技术的部分: ①代换密码; ②换位密码; ③弗纳姆密码。
8. AES 的分组长度和密钥长度是多少? AES 的引入对密码学带来的积极影响有哪些?
9. 为什么 AES 被认为是非常有效的? 在 AES 的实现中, 有限域 F_{2^8} 中的乘法是如何实现的?
10. 在分组密码的密码分组链接 (CBC) 运行模式下, 如果收到的密文的解密“具有正确的填充”, 你认为传输的明文有有效的数据完整性吗?
11. 为什么祖冲之密码算法在完成初始化进入工作状态后, 将算法第一次执行过程 F 的输出 W 舍弃?
12. 试从算法角度, 对 SM4 与 AES 进行比较。